EWB COURSES | AWS ARCHITECT ASSOCIATE | STUDENT PROJECT 01

PROJECT REPORT

AWS VPC Peering:

Establishing Private Connectivity Between Two Amazon VPCs

Amazon VPC · VPC Peering · EC2 · Route Tables · Security Groups · Bastion Host · SSH ProxyJump

Student name	Trisha
Batch	AWS Internship Batch 1
Course / subject	AWS Architect Associate (EWB Courses): AWS Networking, Project 01: VPC Peering
AWS Region	Sydney (ap-southeast-2)
Completion / submission date	26 October 2026

Table of Contents

Abstract 4
1. Introduction 4
1.1 Amazon VPC 4
1.2 Purpose of VPC Peering 4
1.3 Why Private Connectivity Is Useful 4
2. Problem Statement 5
3. Objectives 5
4. Scope and Requirements 5
4.1 Hardware and Software Requirements 5
4.2 AWS Services Used 5
4.3 SSH Client and Terminal Requirements 6
4.4 Out of Scope 6
5. Technical Background 6
5.1 VPCs and CIDR Blocks 6
5.2 Public versus Private Subnets 6
5.3 Route Tables and Local Routes 6
5.4 Internet Gateway 7
5.5 VPC Peering 7
5.6 Security Groups 7
5.7 Bastion Host and SSH ProxyJump 7
5.8 ICMP and Ping 7
6. System Architecture 8
6.1 Reading the Diagram 8
7. IP Addressing and Resource Inventory 8
7.1 Network Resources 9
7.2 EC2 Instances 9
7.3 Known Facts versus Missing Information 11
8. Implementation Methodology 11
8.1 Implementation Sequence 11
9. Route Table Configuration 13
9.1 ewb-rt-a-public (rtb-05928fa32e5e99a75) 13
9.2 ewb-rt-a-private (rtb-0e32ae5a45c32e421) 14
9.3 ewb-rt-b-private (rtb-019b95b5bfdda447a) 14
9.4 Subnet Associations 14
9.5 Why Both Forward and Return Routes Are Needed 15
10. Security Group Configuration 15
10.1 Principle of Least Privilege 16

11. SSH Access and Bastion Host 17
11.1 The SSH Path 17
11.2 How ProxyJump Works Here 17
11.3 Structure of the SSH Configuration 17
12. Testing and Validation 18
12.1 Test Evidence 20
13. Packet Flow Explanation 21
13.1 Forward Path: App-A (10.10.2.10) → App-B (10.20.1.10) 21
13.2 Return Path: App-B → App-A 21
13.3 Why Both Sides Need Routing and Security Rules 21
14. Troubleshooting and Observations 22
14.1 Controlled ICMP Failure and Recovery 22
15. Results and Discussion 22
16. Limitations 22
17. Security Considerations 23
18. Cleanup and Cost Management 23
18.1 Cost Management Notes 25
19. Conclusion and What I Learned 25
20. References 25
Appendix A. Evidence Index 26
Appendix B. Workbook Submission Checklist Mapping 26

Abstract

Cloud workloads are frequently split across separate virtual networks for isolation, ownership or lifecycle reasons, yet they still need to exchange traffic without crossing the public internet. This project demonstrates private connectivity between two Amazon Virtual Private Clouds (VPCs) in the AWS Sydney Region (ap-southeast-2) using VPC peering. VPC A (10.10.0.0/16) contains a public bastion host and a private application instance, App-A (10.10.2.10). VPC B (10.20.0.0/16) contains a single private application instance, App-B (10.20.1.10). Neither application instance has a public IPv4 address, and VPC B has no Internet Gateway.

A peering connection (ewb-a-b-peer) between the two VPCs was activated, routes were added in the relevant route tables on both sides, and security groups were scoped to individual lab hosts using /32 sources. Administration used SSH ProxyJump through the bastion so that no private key is stored on any instance. Testing showed successful SSH to App-A and App-B, successful bidirectional ICMP ping between the private addresses (4 of 4 packets in each direction, 0% loss), a controlled failure created by removing one inbound ICMP security group rule (4 of 4 packets lost), and full recovery after the rule was restored.

The report documents the architecture, resource inventory, route tables and security groups, packet flow, test results, and cleanup evidence. It states plainly which evidence was not captured, most importantly the pre-peering baseline failure, and distinguishes completed lab-network cleanup from items not independently evidenced.

1. Introduction

1.1 Amazon VPC

Amazon Virtual Private Cloud (VPC) lets a customer define a logically isolated virtual network inside an AWS Region. The customer chooses the IPv4 address range (a CIDR block), divides it into subnets, controls routing with route tables, and controls traffic to individual resources with security groups. Resources such as EC2 instances are launched into subnets and receive private IP addresses from the subnet's range. A VPC behaves like a traditional private data-centre network, but its building blocks are created and changed through software in minutes.

1.2 Purpose of VPC Peering

A VPC peering connection is a networking link between two VPCs that allows resources in either VPC to communicate using private IP addresses, as if they were on the same network. Traffic stays on the AWS network and does not traverse the public internet. Peering does not require an Internet Gateway, a VPN or a separate appliance. The link by itself does not move any traffic: each VPC must also have route table entries that send traffic for the other VPC's CIDR to the peering connection, and security rules at the destination must allow the traffic.

1.3 Why Private Connectivity Is Useful

- **Reduced exposure:** servers that never receive a public IP address cannot be reached directly from the internet, which shrinks the attack surface.
- **Cost and simplicity:** private traffic between peered VPCs avoids the need for public IPs, NAT gateways or VPN tunnels for this use case.
- **Performance and predictability:** traffic uses the AWS network rather than the public internet, so latency is low and consistent (sub-millisecond round-trip times were observed in this project).
- **Separation of duties:** teams or workloads can keep separate VPCs (separate routing and security policy) and still share specific services deliberately.

2. Problem Statement

An application server in VPC A and a second workload server in VPC B must both remain private, with no public IPv4 addresses, yet they must exchange traffic using their private IPv4 addresses. Administrators also need a safe way to reach both servers. The problem is therefore to (a) connect two non-overlapping VPCs privately, (b) configure routing so that both the request path and the reply path exist, (c) permit only the minimum inbound traffic required, (d) provide controlled administrative access through a single public bastion in VPC A, and (e) *prove* the result with real test evidence, including a controlled failure that shows which control is responsible for blocking or permitting the traffic.

3. Objectives

- Plan non-overlapping VPC CIDRs (10.10.0.0/16 and 10.20.0.0/16) and /24 subnets for the two VPCs.
- Create public and private subnets with explicit route table associations, one Internet Gateway (VPC A only) and no Internet Gateway in VPC B.
- Launch a bastion (public) and two application instances (private, no public IPv4) with fixed private addresses.
- Request, accept and activate a VPC peering connection, then configure the forward and return routes.
- Restrict inbound security group rules to the specific lab hosts (SSH and ICMP, /32 sources).
- Reach both private servers over SSH through the bastion using ProxyJump, without copying private keys onto any instance.
- Verify private connectivity in both directions with ping, demonstrate a controlled failure and recovery, and explain the forward and return packet paths.
- Clean up the lab resources after evidence is saved (completed for the lab network and instances; see Section 18 for evidence and verification caveats).

4. Scope and Requirements

4.1 Hardware and Software Requirements

Item	Requirement / what was used	Basis
Client computer	A laptop with internet access and a modern web browser for the AWS Management Console. The terminal screenshots show a Windows command prompt.	**EVIDENCED**
SSH client	OpenSSH client (ssh) with a per-host configuration file. Windows OpenSSH with ProxyJump was used. The workbook also permits WSL, macOS or Linux.	**PROJECT RECORD**
Operating system on instances	Amazon Linux 2023 (login banner visible in terminal screenshots); default user ec2-user.	**EVIDENCED**
Instance type / Availability Zone	t3.micro for all three instances, all in ap-southeast-2a.	**EVIDENCED**
AWS account permissions	Permission to manage VPC, EC2 and key pairs in a single account and single Region.	Workbook

4.2 AWS Services Used

Service / feature	Role in this project
Amazon VPC	Two isolated virtual networks (VPC A and VPC B) with their CIDR blocks, subnets and route tables.

Service / feature	Role in this project
VPC peering connection	Private, one-to-one link between VPC A and VPC B (ewb-a-b-peer).
Internet Gateway	Provides internet entry for the bastion in VPC A only (ewb-igw-a).
Amazon EC2	Bastion and two application instances (Amazon Linux 2023, t3.micro).
Security groups	Stateful instance-level firewalls restricting inbound SSH and ICMP to specific /32 sources.
EC2 key pair	RSA key pair used for SSH authentication; the private key stays on the laptop. The workbook names the key pair ewb-lab.

4.3 SSH Client and Terminal Requirements

SSH commands run on the laptop, not on the instances. The key file must be readable only by its owner (on Linux, macOS and WSL this is `chmod 400`; on Windows the file's permissions must be restricted to the current user, otherwise OpenSSH refuses to use it). The SSH configuration defines three host aliases (`ewb-bastion`, `ewb-app-a`, `ewb-app-b`); the two private aliases use `ProxyJump ewb-bastion`. Section 11 shows the structure of this configuration.

4.4 Out of Scope

NAT gateways, VPC endpoints, Transit Gateway, cross-account or cross-Region peering, IPv6, custom network ACLs and peering DNS-resolution options are outside the scope of this project. The lab uses the default allow-all network ACLs.

5. Technical Background

5.1 VPCs and CIDR Blocks

A CIDR block such as 10.10.0.0/16 identifies a range of IPv4 addresses: the first 16 bits are the network prefix and the remaining 16 bits identify hosts, giving 65,536 addresses (10.10.0.0 to 10.10.255.255). The second VPC uses 10.20.0.0/16. These two ranges do not overlap, which is a hard requirement for peering: AWS will not create a peering connection between VPCs whose CIDR blocks overlap, and different subnet ranges cannot fix overlapping VPC ranges. A /24 subnet contains 256 addresses; AWS reserves five of them in every subnet (the first four and the last), leaving 251 usable. A subnet lives entirely within one Availability Zone, and every subnet CIDR must fall inside its VPC's CIDR.

5.2 Public versus Private Subnets

The difference between a public and a private subnet is routing, not a setting on the subnet itself. A **public subnet** is associated with a route table that has a route to an Internet Gateway (0.0.0.0/0 → igw). A **private subnet** has no such route. For an instance to be reachable directly from the internet it needs both: a subnet with an Internet Gateway route and a public IPv4 address. In this project only the bastion has a public IPv4 address; App-A and App-B are in subnets without an internet route and have no public IPv4 address at all.

5.3 Route Tables and Local Routes

Every subnet is associated with exactly one route table, and the route table decides where packets leaving the subnet are sent. Each route table automatically contains a **local route** for the VPC's own CIDR (for example 10.10.0.0/16 → local), which provides connectivity between all subnets in that VPC and cannot be deleted. When several routes match a destination, AWS selects the most specific one (longest prefix match), so a route to 10.20.0.0/16 wins over 0.0.0.0/0 for traffic addressed to VPC B. This project uses explicit subnet associations with three custom route tables rather than relying on the main route table.

5.4 Internet Gateway

An Internet Gateway (IGW) is a horizontally scaled, redundant and highly available VPC component that allows communication between instances in a VPC and the internet. An IGW is attached to one VPC and is used as a *target* in route tables. For IPv4 it performs one-to-one network address translation between an instance's private IP and its public IP. Here ewb-igw-a is attached to VPC A only; VPC B deliberately has none.

5.5 VPC Peering

A VPC peering connection is created by a *requester* VPC and must be *accepted* by the owner of the *accepter* VPC, even when both belong to the same account. Once the status is Active, traffic can flow, but only if routes exist on both sides. Key properties relevant to this project:

- Peering is **one-to-one and non-transitive**: if A is peered with B and B with C, A cannot reach C through B.
- Peering does **not** create routes automatically; each VPC's route tables must be updated.
- Peering does **not** provide edge-to-edge routing: a VPC cannot use its peer's Internet Gateway, NAT device or VPN. VPC B therefore gains no internet access through VPC A.
- The CIDR blocks of the two VPCs (including any secondary CIDRs) must not overlap.
- Peering traffic uses AWS's network infrastructure; it is not a gateway or VPN and has no separate bandwidth bottleneck or single point of failure.

5.6 Security Groups

A security group is a virtual firewall attached to an instance's network interface. Rules are **allow-only**: any inbound traffic not matched by a rule is dropped. Security groups are **stateful**, so reply traffic for an allowed request is automatically permitted regardless of inbound rules. A source written as 10.10.2.10/32 means exactly one IPv4 address. By default a new security group allows all outbound traffic. Network ACLs, in contrast, are stateless and apply per subnet; this lab leaves them at the default allow-all setting. The principle of least privilege (grant only the access required) is applied by using individual /32 sources instead of broad ranges.

5.7 Bastion Host and SSH ProxyJump

A bastion host is a deliberately exposed, tightly restricted server that acts as the single administrative entry point to otherwise private servers. With OpenSSH ProxyJump (equivalent to ssh -J), the client first connects to the bastion, asks it to forward a TCP connection to the target's port 22, and then performs the SSH handshake and key authentication with the target *through* that forwarded connection. The private key therefore never leaves the laptop and is never copied to the bastion. Because the bastion opens the TCP connection to the target, the target's security group sees the **bastion's private IP** (10.10.1.10) as the source, which is why the SSH rules on App-A and App-B use 10.10.1.10/32.

5.8 ICMP and Ping

The Internet Control Message Protocol (ICMP) is a network-layer protocol (IP protocol number 1) used for diagnostics and error reporting; it has no ports. The ping utility sends ICMP Echo Request messages (type 8, code 0) and waits for Echo Reply messages (type 0). The command ping -c 4 <ip> sends four requests and prints packets transmitted and received, percentage loss and round-trip time statistics. A successful ping shows that routing in both directions and the ICMP security rule at the destination are working; it does not by itself prove that other protocols (such as TCP 22) are permitted.

6. System Architecture

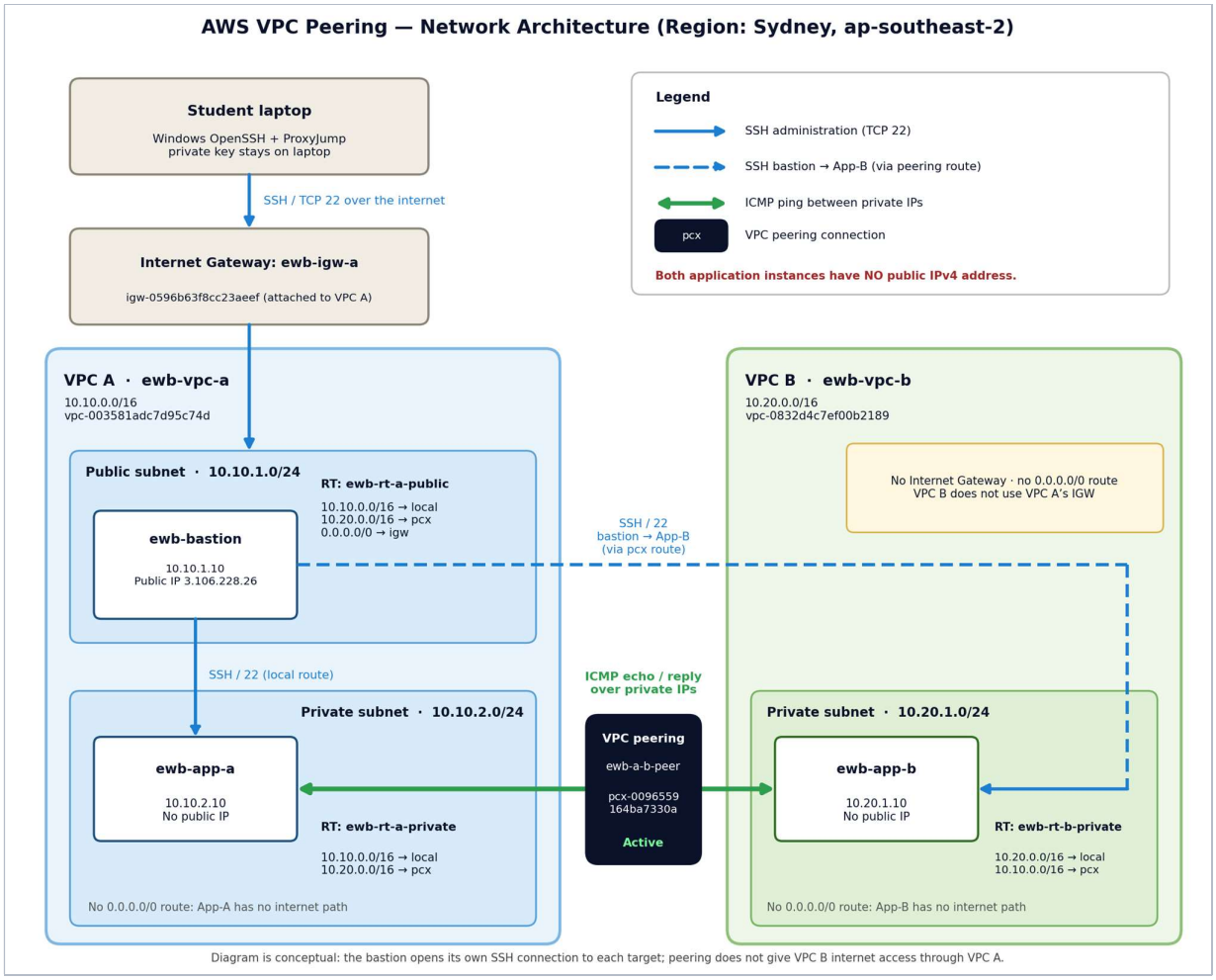


Figure 1. *Network architecture: laptop → bastion over SSH; bastion → App-A and App-B (SSH); App-A ⇄ App-B over private IPs through the peering connection. App-A and App-B have no public IPv4 address, and VPC B has no Internet Gateway.*

6.1 Reading the Diagram

- **Administration path:** laptop → internet → ewb-igw-a → bastion (10.10.1.10, public IP 3.106.228.26 at test time) → App-A or App-B using SSH ProxyJump. The bastion opens a separate SSH connection to each destination.
- **Application traffic path:** App-A (10.10.2.10) ⇄ App-B (10.20.1.10) over private IPs through pcx-0096559164ba7330a. It never touches the Internet Gateway.
- **Isolation:** App-A and App-B have no public IPv4 address. Their route tables have no 0.0.0.0/0 route. VPC B has no Internet Gateway at all and cannot borrow VPC A's, because peering does not provide edge-to-edge routing.
- **Why the public route table also has a peering route:** the bastion's SSH connection to App-B originates in the public subnet, so ewb-rt-a-public must know how to reach 10.20.0.0/16. The more specific peering route is chosen over the internet default route.

7. IP Addressing and Resource Inventory

Every ID below was read from the supplied screenshots unless marked otherwise. "Project record" means the value was provided in the project configuration notes but is not visible in a screenshot. "Not available"

means the information is neither in the screenshots nor in the project record, so it has been left blank rather than guessed.

7.1 Network Resources

Resource	Name	ID	CIDR / address	Basis
VPC A	ewb-vpc-a	vpc-003581adc7d95c74d	10.10.0.0/16	**EVIDENCED** Figs. 2, 3, 5
VPC B	ewb-vpc-b	vpc-0832d4c7ef00b2189	10.20.0.0/16	**EVIDENCED** Figs. 4, 5
Public subnet (VPC A)	ewb-a-public	available	10.10.1.0/24	**PROJECT RECORD**
Private subnet (VPC A)	ewb-a-private	available	10.10.2.0/24	**PROJECT RECORD**
Private subnet (VPC B)	ewb-b-private	available	10.20.1.0/24	**PROJECT RECORD**
Internet Gateway	ewb-igw-a	igw-0596b63f8cc23aeef	Attached to VPC A	**EVIDENCED** ID seen as route target, Fig. 6
Route table (public)	ewb-rt-a-public	rtb-05928fa32e5e99a75	VPC A	**EVIDENCED** Fig. 6
Route table (private A)	ewb-rt-a-private	rtb-0e32ae5a45c32e421	VPC A	**EVIDENCED** Fig. 7
Route table (private B)	ewb-rt-b-private	rtb-019b95b5bfdda447a	VPC B	**EVIDENCED** Fig. 8
Peering connection	ewb-a-b-peer	pcx-0096559164ba7330a	Requester VPC A ⇄ Accepter VPC B; Active	**EVIDENCED** Fig. 5
Security group	ewb-sg-bastion	sg-0725cba0ffaaefc14	VPC A	**EVIDENCED** Fig. 9
Security group	ewb-sg-app-a	sg-02e3f9a604ebb0522	VPC A	**EVIDENCED** Fig. 10
Security group	ewb-sg-app-b	sg-0ce9e1b7264e57a37	VPC B	**EVIDENCED** Fig. 11

7.2 EC2 Instances

Name	Instance ID	Subnet / VPC	Private IP	Public IPv4	AZ / type	Security group
ewb-bastion	i-047f321971d45e684	ewb-a-public / VPC A	10.10.1.10	3.106.228.26 (auto-assigned; no Elastic IP)	ap-southeast-2a / t3.micro	ewb-sg-bastion
ewb-app-a	i-0170f18f690c0ba41	ewb-a-private / VPC A	10.10.2.10	None (field shows “–”)	ap-southeast-2a / t3.micro	ewb-sg-app-a

Name	Instance ID	Subnet / VPC	Private IP	Public IPv4	AZ / type	Security group
ewb-app-b	i-0b8529554ab35eb8d	ewb-b-private / VPC B	10.20.1.10	None (field shows "–")	ap-southeast-2a / t3.micro	ewb-sg-app-b

Instance IDs, VPC IDs, private IPs, public IPv4 state, AZ, instance type and 3/3 status checks are read from Figures 2–4. The security group names for each instance and the subnet names are from the project record and workbook; the instance detail screenshots do not show them. The subnet-to-instance mapping is also consistent with the private IPs (10.10.1.10 in 10.10.1.0/24, and so on).

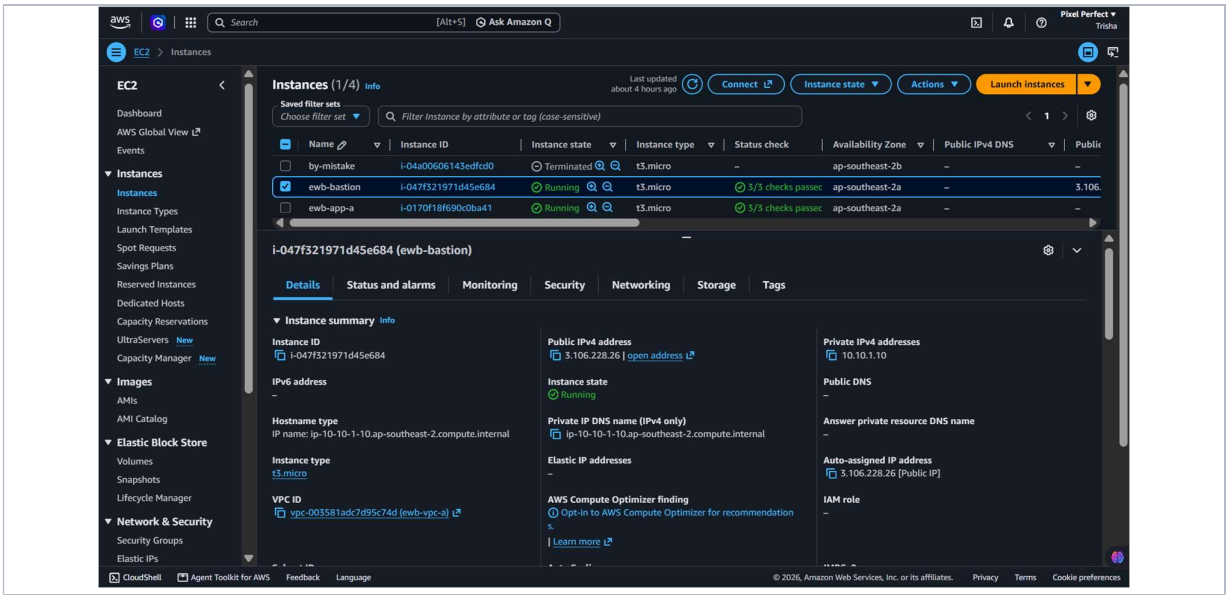


Figure 2. ewb-bastion *| EC2 console | Instance i-047f321971d45e684 running, private IP 10.10.1.10, public IPv4 3.106.228.26, VPC ewb-vpc-a, t3.micro, ap-southeast-2a, 3/3 checks passed. The same list shows an unrelated terminated instance named "by-mistake".*

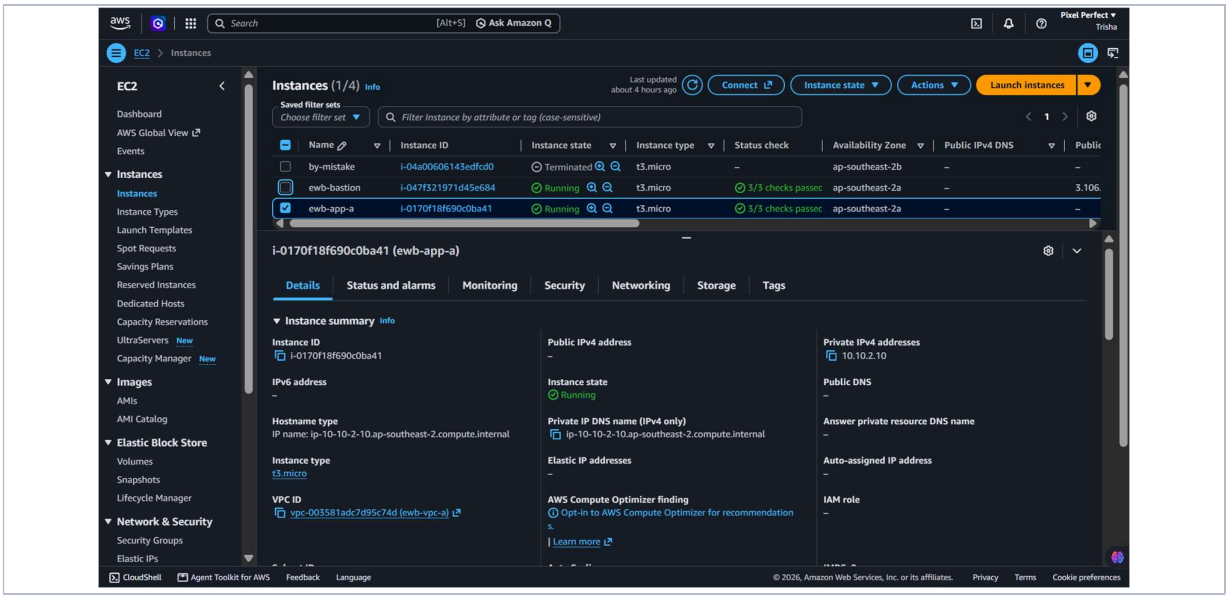


Figure 3. ewb-app-a *| EC2 console | Instance i-0170f18f690c0ba41 running, private IP 10.10.2.10,* ***Public IPv4 address: "–" (none)****, VPC ewb-vpc-a.*

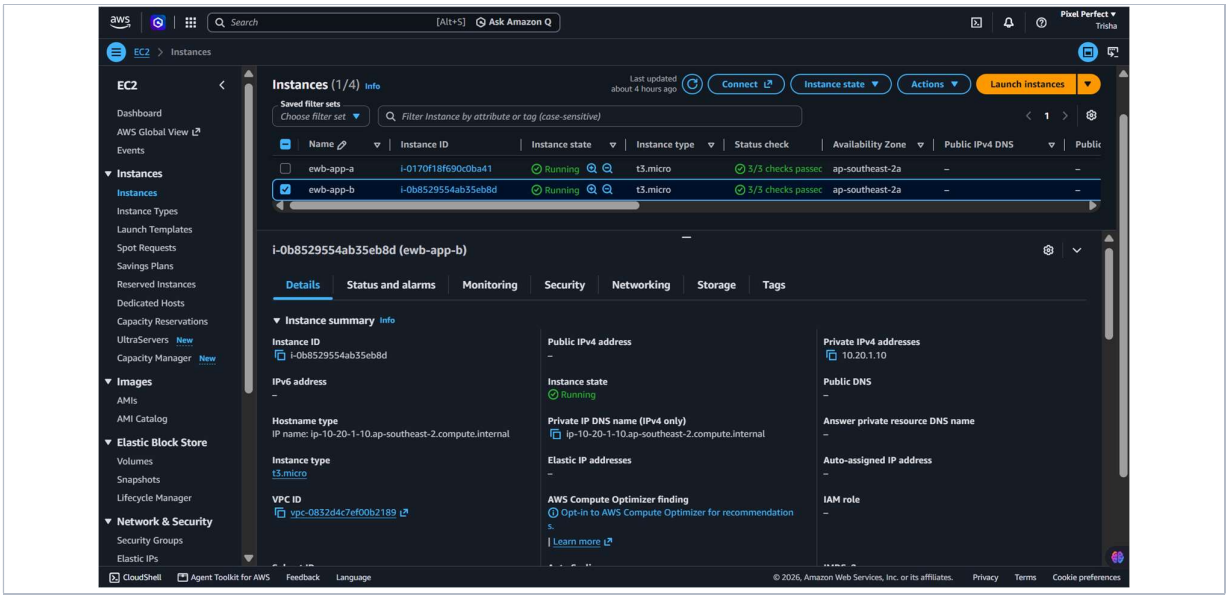


*Figure 4. **ewb-app-b** | EC2 console | Instance i-0b8529554ab35eb8d running, private IP 10.20.1.10, **Public IPv4 address: "–" (none)**, VPC vpc-0832d4c7ef00b2189 (ewb-vpc-b).*

7.3 Known Facts versus Missing Information

Information	Status
VPC IDs, peering ID, IGW ID, route table IDs, security group IDs and rules, instance IDs, private/public IP state, AZ, instance type	**EVIDENCED**
Region, CIDRs, resource names, bastion public IP at test time, SSH/ping outcomes, controlled failure and recovery	**PROJECT RECORD** with matching screenshots where cited
Subnet IDs	**EVIDENCED**
Subnet associations of the three route tables (workbook Checkpoints 01 and 04)	EVIDENCE SUPPLIED in Appendix C (association screenshots)
AWS cleanup of any resource	LAB NETWORK CLEANUP **EVIDENCED**

8. Implementation Methodology

The procedure followed the EWB workbook (pages 5 to 10). The table gives each stage, the configuration used, and the evidence that supports it. Where no screenshot was supplied the evidence column says so; the configuration is then taken from the project record.

8.1 Implementation Sequence

#	Stage	What was configured	Evidence
1	VPC creation	VPC A ewb-vpc-a (10.10.0.0/16) and VPC B ewb-vpc-b (10.20.0.0/16), created as "VPC only" with default tenancy and no IPv6, in ap-southeast-2.	VPC IDs and CIDRs: Figs. 2–5. Creation screens: **AVAILABLE**

#	Stage	What was configured	Evidence
2	Subnet creation	VPC A: ewb-a-public 10.10.1.0/24 and ewb-a-private 10.10.2.0/24. VPC B: ewb-b-private 10.20.1.0/24. Public IPv4 auto-assign disabled on the private subnets.	CIDRs from project record. Subnet IDs and screens: **AVAILABLE**
3	Internet Gateway	ewb-igw-a created and attached to ewb-vpc-a only. VPC B has no Internet Gateway.	IGW ID as route target: Fig. 6
4	Route tables and associations	Three custom route tables: ewb-rt-a-public (associated with ewb-a-public), ewb-rt-a-private (ewb-a-private), ewb-rt-b-private (ewb-b-private). Local routes retained.	Routes: Figs. 6–8. Associations: **AVAILABLE**
5	EC2 deployment	Amazon Linux 2023, t3.micro, same AZ. Bastion: 10.10.1.10 with public IPv4. App-A: 10.10.2.10, App-B: 10.20.1.10, both with public IPv4 disabled.	Figs. 2–4
6	Security groups	ewb-sg-bastion: SSH from operator /32. ewb-sg-app-a and ewb-sg-app-b: SSH from 10.10.1.10/32 and ICMP from the peer instance /32.	Figs. 9–11
7	Bastion setup	Bastion in the public subnet with the Internet Gateway route and an auto-assigned public IPv4; key pair stored only on the laptop.	Fig. 2, Fig. 12
8	SSH configuration	Local SSH config with aliases ewb-bastion, ewb-app-a, ewb-app-b; the two private aliases use ProxyJump ewb-bastion (Windows OpenSSH).	Aliases used in Figs. 12–14. Config file: **AVAILABLE**
9	Peering request and acceptance	ewb-a-b-peer: requester ewb-vpc-a, accepter ewb-vpc-b, same account and Region. Request accepted; status Active.	Fig. 5
10	Route configuration on both sides	10.20.0.0/16 → pcx in the two VPC A route tables; 10.10.0.0/16 → pcx in ewb-rt-b-private (return route).	Figs. 6–8
11	Connectivity verification	SSH to both private hosts; ping in both directions; controlled ICMP failure and recovery.	Figs. 12–18

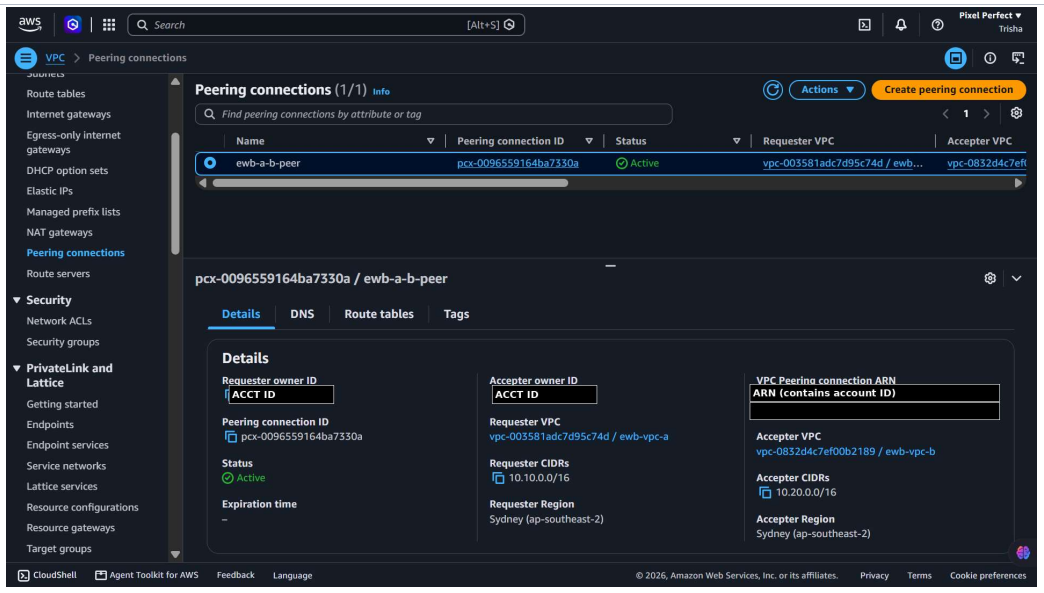


Figure 5. Peering connection *| VPC console | ewb-a-b-peer (pcx-0096559164ba7330a) is* ***Active****; requester ewb-vpc-a 10.10.0.0/16, accepter ewb-vpc-b 10.20.0.0/16, both Sydney (ap-southeast-2). Account IDs and ARN redacted.*

9. Route Table Configuration

Three custom route tables control traffic in the project. The AWS-created local routes were preserved in every table; only the missing routes were added. In the screenshots each route has status Active; local routes show Route Origin *CreateRouteTable* and the added routes show *CreateRoute*.

9.1 ewb-rt-a-public (rtb-05928fa32e5e99a75)

Destination	Target	Purpose	Evidence
10.10.0.0/16	local	Bastion to App-A and all VPC A subnets (AWS-created, preserved).	Fig. 6
10.20.0.0/16	pcx-0096559164ba7330a	Bastion's SSH connection to App-B (ProxyJump) must be routed to VPC B. More specific than the default route, so it is selected.	Fig. 6
0.0.0.0/0	igw-0596b63f8cc23aeef	Internet access for the bastion (SSH from the laptop and replies).	Fig. 6

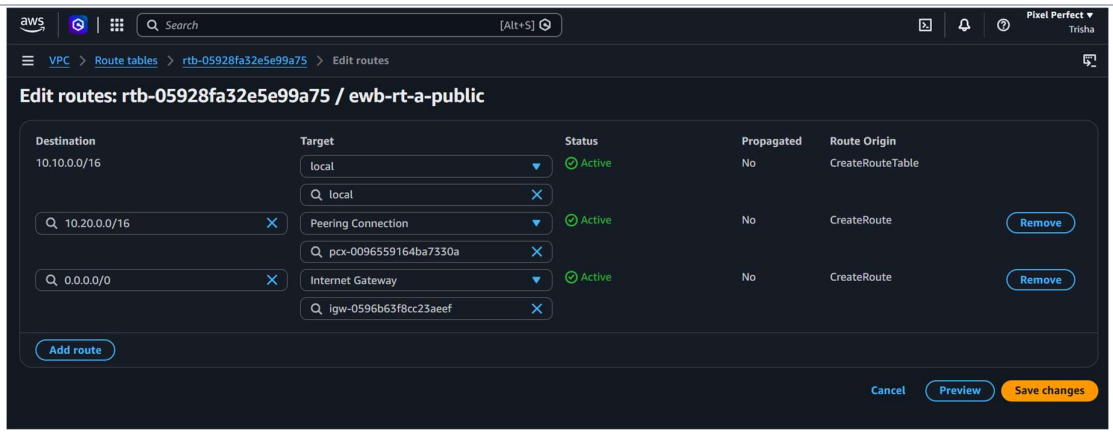


Figure 6. ewb-rt-a-public *| VPC console, Edit routes | local, 10.20.0.0/16 → peering connection, 0.0.0.0/0 → Internet Gateway; all Active.*

9.2 ewb-rt-a-private (rtb-0e32ae5a45c32e421)

Destination	Target	Purpose	Evidence
10.10.0.0/16	local	App-A to other VPC A subnets, including the bastion (AWS-created, preserved).	Fig. 7
10.20.0.0/16	pcx-0096559164ba7330a	**Forward route:** App-A to VPC B (App-B).	Fig. 7

There is deliberately no 0.0.0.0/0 route, so App-A has no internet path.

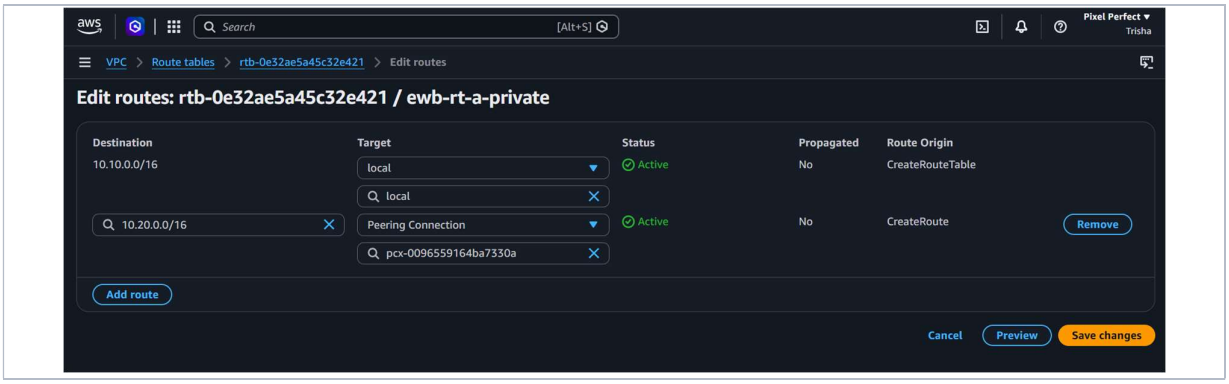


*Figure 7. **ewb-rt-a-private** | VPC console, Edit routes | local and 10.20.0.0/16 → peering connection; both Active; no default route.*

9.3 ewb-rt-b-private (rtb-019b95b5bfdda447a)

Destination	Target	Purpose	Evidence
10.20.0.0/16	local	App-B to other VPC B resources (AWS-created, preserved).	Fig. 8
10.10.0.0/16	pcx-0096559164ba7330a	**Return route:** replies from App-B to VPC A (App-A and the bastion).	Fig. 8

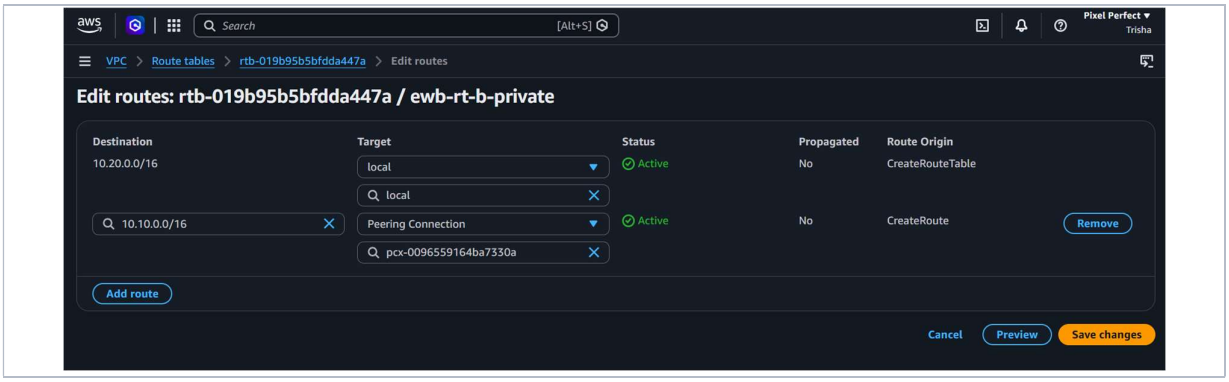


*Figure 8. **ewb-rt-b-private** | VPC console, Edit routes | local and 10.10.0.0/16 → peering connection; both Active; no Internet Gateway route.*

9.4 Subnet Associations

The route editor screenshots confirm each table's routes, but they do not show the "Subnet associations" tab. The intended explicit associations (from the workbook and project record) are listed below. The final column is left for the student to complete if the association screenshot (workbook Checkpoints 01 and 04) is added before submission.

Route table	Intended associated subnet	Subnet ID	Association evidence
ewb-rt-a-public	ewb-a-public (10.10.1.0/24)	Available	Captured
ewb-rt-a-private	ewb-a-private (10.10.2.0/24)	Available	Captured

Route table	Intended associated subnet	Subnet ID	Association evidence
ewb-rt-b-private	ewb-b-private (10.20.1.0/24)	Available	Captured

9.5 Why Both Forward and Return Routes Are Needed

A route is a one-way instruction: it says where *outgoing* packets from a subnet go. When App-A pings App-B, the echo request leaves VPC A using 10.20.0.0/16 → pcx. When App-B answers, the echo reply is a new packet from 10.20.1.10 to 10.10.2.10, and VPC B needs its own route, 10.10.0.0/16 → pcx, to send it back. Without that return route the request would arrive but the reply would have no path, and the ping would still fail. Security groups being stateful does not help here: statefulness lets the reply *through the firewall*, but routing still has to deliver it. The same logic explains why the public subnet needs a route to VPC B for the bastion's SSH connection and why App-B's return route covers all of 10.10.0.0/16 (including the bastion at 10.10.1.10).

10. Security Group Configuration

Security group inbound rules were read from the "Edit inbound rules" screenshots. Outbound rules were not captured; the workbook specifies the default allow-all outbound rule, so none were changed, but this is not shown in the evidence. Because security groups are stateful, no inbound rule is needed for replies.

Security group (ID)	Rule ID	Type / protocol / port	Source	Purpose	Evidence
ewb-sg-bastion sg-0725cba0ffaaefc14	sgr-047a144f34d432d9c	SSH / TCP / 22	Operator's public IPv4 /32 (redacted)	Lets only the administrator's own address reach the bastion.	Fig. 9
ewb-sg-app-a sg-02e3f9a604ebb0522	sgr-08df46aaa22c9dd03	All ICMP – IPv4 / ICMP / All	10.20.1.10/32 (App-B)	Allows App-B's ping to reach App-A (reverse test).	Fig. 10
	sgr-013133691f4301838	SSH / TCP / 22	10.10.1.10/32 (bastion private IP)	Administrative SSH to App-A only from the bastion.	Fig. 10
ewb-sg-app-b sg-0ce9e1b7264e57a37	sgr-02f68656c631c9f11	All ICMP – IPv4 / ICMP / All	10.10.2.10/32 (App-A)	Allows App-A's ping to reach App-B (forward test). This is the rule removed and restored in the controlled failure.	Fig. 11
	sgr-0b2c775d685339e27	SSH / TCP / 22	10.10.1.10/32 (bastion private IP)	Administrative SSH to App-B only from the bastion.	Fig. 11

10.1 Principle of Least Privilege

Least privilege means granting only the access needed to do the job. Here that is applied in four ways: (1) the bastion accepts SSH only from one operator address, not 0.0.0.0/0; (2) each application instance accepts SSH only from the bastion's single private address; (3) ICMP is accepted only from the one peer instance, not from the whole peer VPC; and (4) no rule is open to the internet on the application instances, which also have no public IP. If an address changes (for example the operator's ISP address, or an instance IP), the corresponding rule must be updated.

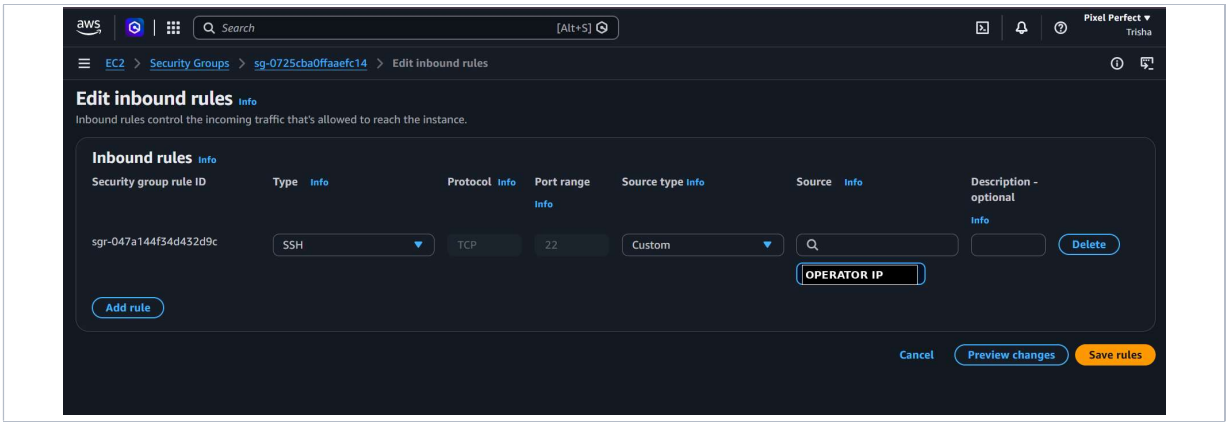


Figure 9. ewb-sg-bastion | *EC2 console, inbound rules | one rule: SSH TCP 22 from a single operator /32 (address redacted).*

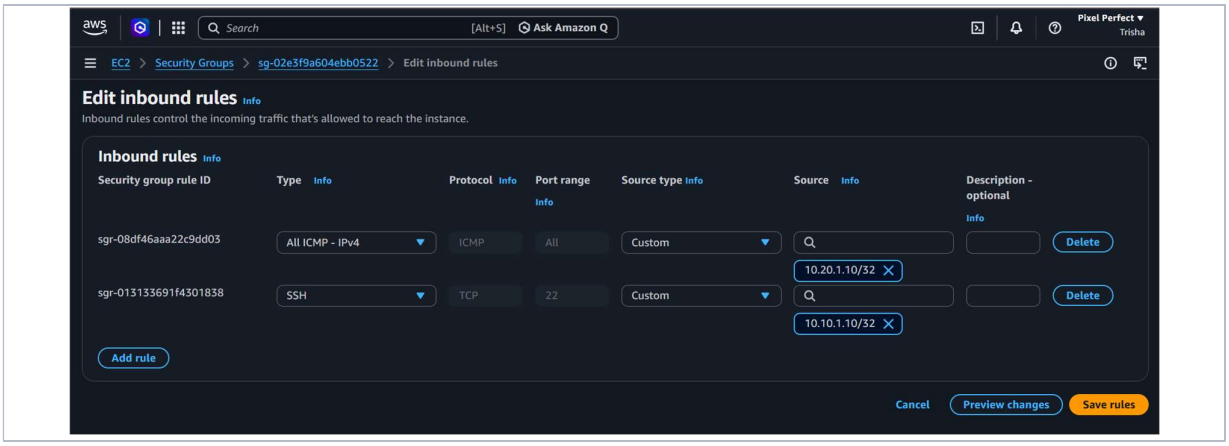


Figure 10. ewb-sg-app-a | *EC2 console, inbound rules | All ICMP–IPv4 from 10.20.1.10/32 and SSH TCP 22 from 10.10.1.10/32.*

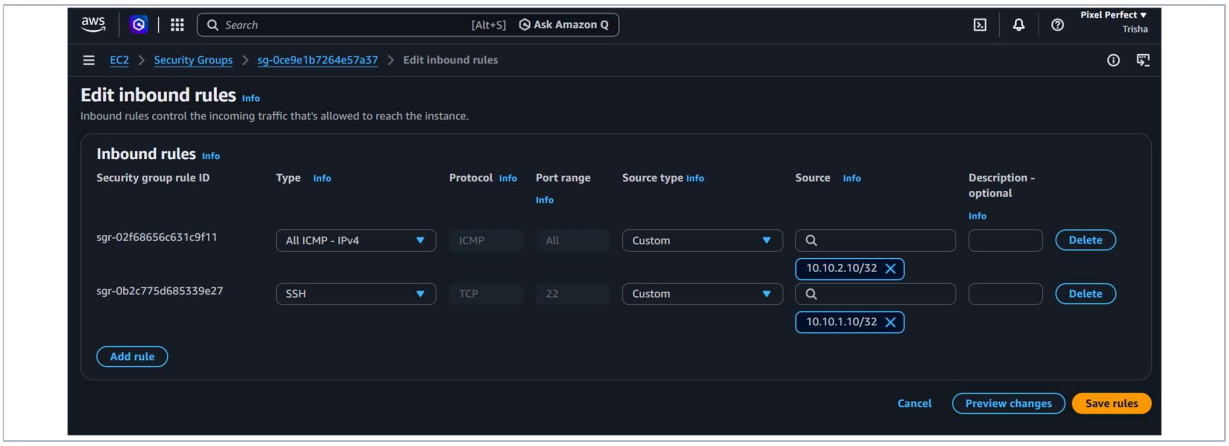


Figure 11. ewb-sg-app-b | *EC2 console, inbound rules | All ICMP–IPv4 from 10.10.2.10/32 and SSH TCP 22 from 10.10.1.10/32.*

11. SSH Access and Bastion Host

11.1 The SSH Path

Hop	From → To	Connection	Allowed by
1	Laptop → bastion public IP (3.106.228.26 at test time)	SSH, TCP 22, over the internet and the Internet Gateway	ewb-sg-bastion (operator /32); ewb-rt-a-public (0.0.0.0/0 → IGW)
2a	Bastion (10.10.1.10) → App-A (10.10.2.10)	SSH, TCP 22, inside VPC A (local route)	ewb-sg-app-a (SSH from 10.10.1.10/32)
2b	Bastion (10.10.1.10) → App-B (10.20.1.10)	SSH, TCP 22, across the peering connection	ewb-rt-a-public (10.20.0.0/16 → pcx); ewb-rt-b-private (10.10.0.0/16 → pcx); ewb-sg-app-b (SSH from 10.10.1.10/32)

11.2 How ProxyJump Works Here

The laptop runs ssh ewb-app-a (or ewb-app-b). OpenSSH reads the host entry, sees ProxyJump ewb-bastion, and first opens an SSH session to the bastion. Over that session it requests a forwarded TCP connection to the target's private IP on port 22, and then runs a second, end-to-end SSH handshake with the target inside that tunnel. The private key is used on the laptop for both hops and is never copied to any instance. The bastion only relays encrypted bytes; the session with App-A or App-B is encrypted end to end. The terminal banners show Last login … from 10.10.1.10 on both private hosts, which is consistent with connections arriving from the bastion's private address, and the bastion's own banner shows the previous login coming from the operator's (redacted) public address.

11.3 Structure of the SSH Configuration

The actual configuration file was not supplied. The structure below follows the workbook template; the placeholders must be replaced locally, and no key material is included. Only the *path* to the key file appears.

```
Host ewb-bastion
   HostName <BASTION_PUBLIC_IP>
   User ec2-user
   IdentityFile ~/.ssh/<lab-key>.pem
   IdentitiesOnly yes

Host ewb-app-a
   HostName 10.10.2.10
   User ec2-user
   IdentityFile ~/.ssh/<lab-key>.pem
   IdentitiesOnly yes
   ProxyJump ewb-bastion

Host ewb-app-b
   HostName 10.20.1.10
   User ec2-user
   IdentityFile ~/.ssh/<lab-key>.pem
   IdentitiesOnly yes
   ProxyJump ewb-bastion
```

Because the bastion's public IP was auto-assigned (no Elastic IP was shown), it changes if the instance is stopped and started; HostName and the operator-side security rule must then be refreshed.

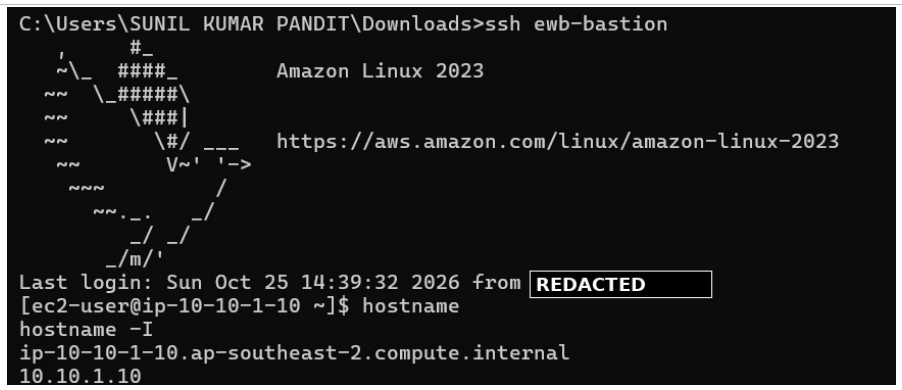


Figure 12. Laptop → ewb-bastion | *Windows terminal | ssh ewb-bastion succeeded; hostname ip-10-10-1-10.ap-southeast-2.compute.internal, hostname -I = 10.10.1.10. Previous-login source IP redacted.*



Figure 13. Laptop → ewb-app-a via bastion | *Windows terminal | ssh ewb-app-a succeeded; hostname ip-10-10-2-10…, hostname -I = 10.10.2.10; last login from 10.10.1.10 (bastion).*

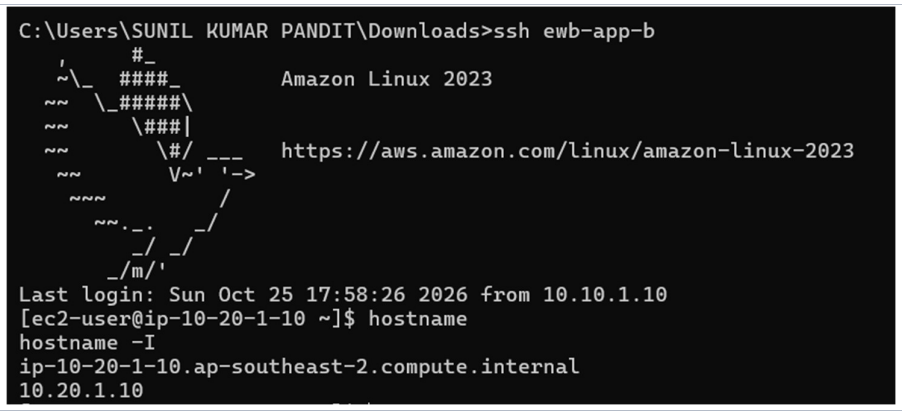


Figure 14. Laptop → ewb-app-b via bastion | *Windows terminal | ssh ewb-app-b succeeded; hostname ip-10-20-1-10…, hostname -I = 10.20.1.10; last login from 10.10.1.10 (bastion).*

12. Testing and Validation

The table compares the workbook's expected result with the actual result read from the terminal output. "Pass" means the observed result matched the expectation; for the controlled-failure row the expectation is the failure itself.

Test objective	Source	Destination	Command / action	Expected	Actual	Status	Evidence

Test objective	Source	Destination	Command / action	Expected	Actual	Status	Evidence
SSH to App-A through the bastion	Laptop	App-A 10.10.2.10	ssh ewb-app-a	Shell on App-A	Amazon Linux 2023 shell; hostname ip-10-10-2-10; 10.10.2.10	**PASS**	Fig. 13
SSH to App-B through the bastion	Laptop	App-B 10.20.1.10	ssh ewb-app-b	Shell on App-B	Shell; hostname ip-10-20-1-10; 10.20.1.10	**PASS**	Fig. 14
Forward ping (after routes)	App-A	10.20.1.10	ping -c 4 10.20.1.10	4 replies; 0% loss	4 sent, 4 received, 0% loss; rtt min/avg/max 0.186/0.218/0.247 ms	**PASS**	Fig. 15
Reverse ping (after routes)	App-B	10.10.2.10	ping -c 4 10.10.2.10	4 replies; 0% loss	4 sent, 4 received, 0% loss; rtt min/avg/max 0.175/0.216/0.293 ms	**PASS**	Fig. 16
Controlled ICMP failure	App-A	10.20.1.10	Inbound ICMP rule for 10.10.2.10/32 removed from ewb-sg-app-b; ping -c 4 10.20.1.10	No replies	4 sent, 0 received, 100% loss (3160 ms)	**PASS** (failure observed)	Fig. 17
Recovery after restoring the rule	App-A	10.20.1.10	Rule restored on ewb-sg-app-b; ping repeated	4 replies; 0% loss	4 sent, 4 received, 0% loss; rtt min/avg/max 0.211/0.227/0.251 ms	**PASS**	Fig. 18
Isolation: no public IPv4 on application hosts	EC2 console	App-A, App-B	Instance details	No public IPv4	Public IPv4 address field is "–" for both	**PASS**	Figs. 3, 4

12.1 Test Evidence

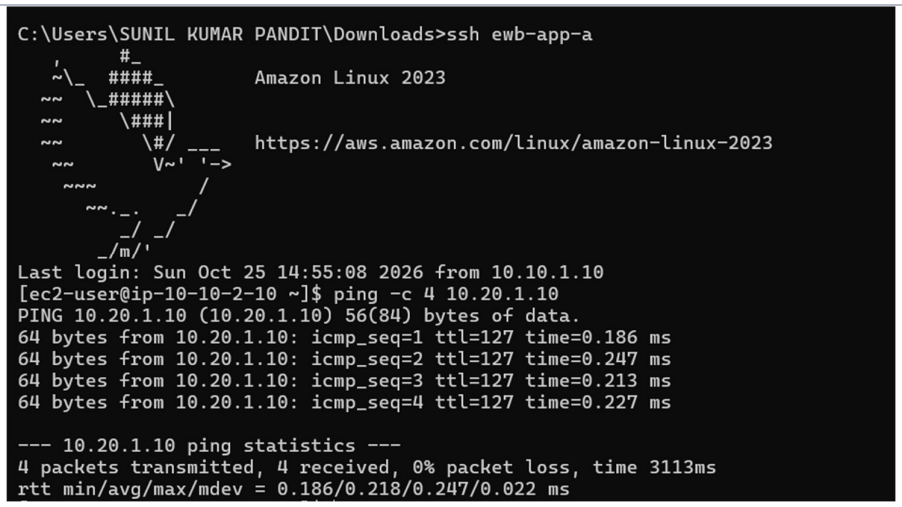


Figure 15. Forward ping *| Host: ewb-app-a (via bastion) | Test: ping -c 4 10.20.1.10 | Result: 4 transmitted, 4 received, 0% packet loss.*

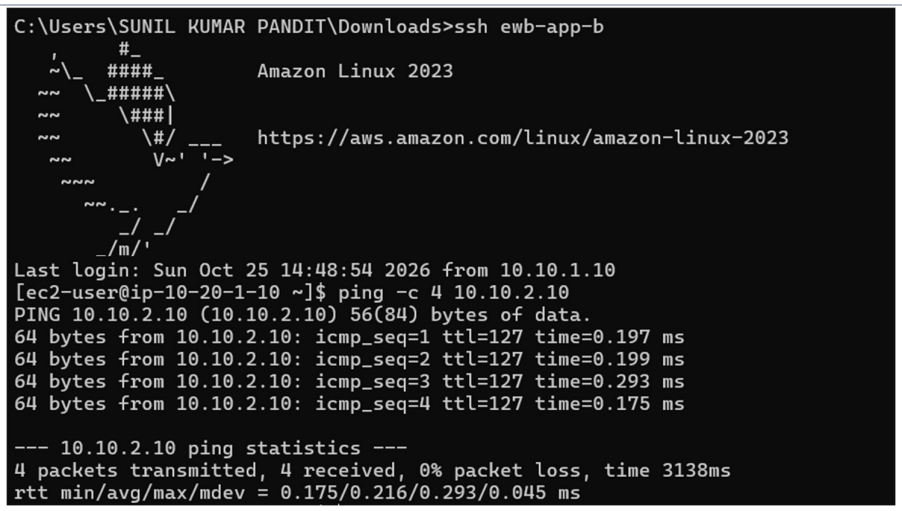


Figure 16. Reverse ping *| Host: ewb-app-b (via bastion) | Test: ping -c 4 10.10.2.10 | Result: 4 transmitted, 4 received, 0% packet loss.*

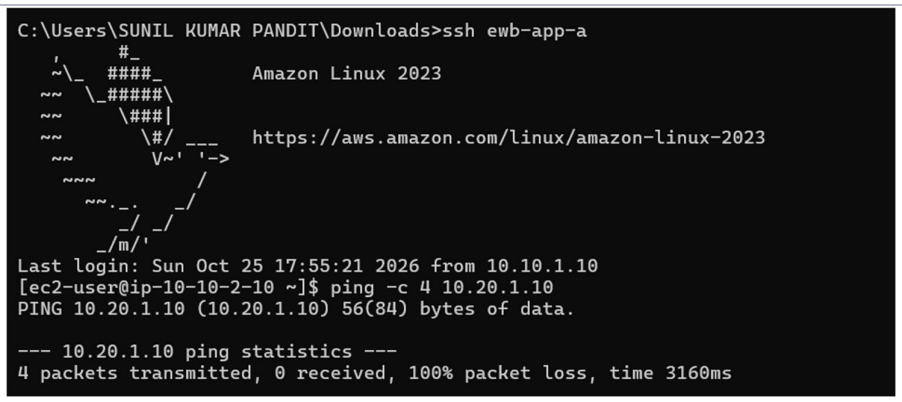


Figure 17. Controlled ICMP failure *| Host: ewb-app-a | Test: ping -c 4 10.20.1.10 with the ICMP rule removed from ewb-sg-app-b | Result: 4 transmitted, 0 received, 100% packet loss.*

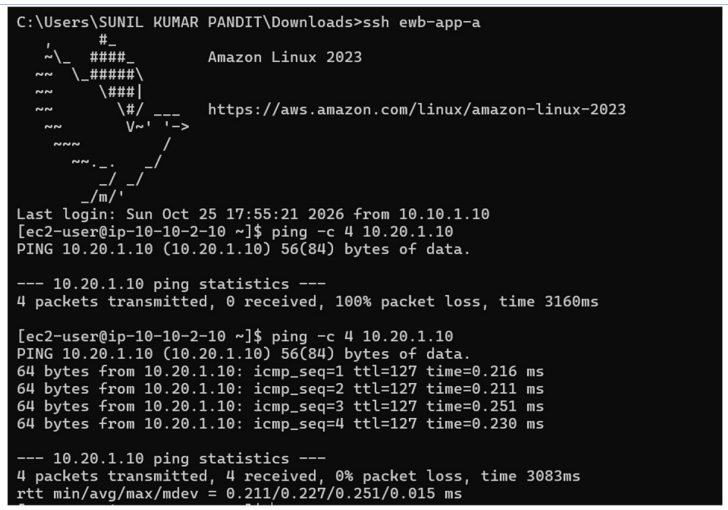


Figure 18. Recovery *| Host: ewb-app-a | The same session shows the failed ping (100% loss) followed by the repeated ping after the rule was restored: 4 transmitted, 4 received, 0% packet loss.*

13. Packet Flow Explanation

13.1 Forward Path: App-A (10.10.2.10) → App-B (10.20.1.10)

1. App-A creates an ICMP Echo Request addressed to 10.20.1.10. The address is outside App-A's own subnet, so the packet is handed to the VPC router.
2. The VPC router consults ewb-rt-a-private. Two routes could match: 10.10.0.0/16 → local does not match, and 10.20.0.0/16 → pcx-0096559164ba7330a does. The packet is sent to the peering connection.
3. The peering connection carries the packet over the AWS network into VPC B. No Internet Gateway is involved.
4. At App-B's network interface the security group ewb-sg-app-b is evaluated. The inbound ICMP rule with source 10.10.2.10/32 matches, so the request is permitted. (The default network ACLs allow all traffic.)
5. App-B's operating system generates an Echo Reply.

13.2 Return Path: App-B → App-A

1. The Echo Reply (source 10.20.1.10, destination 10.10.2.10) is a new packet leaving VPC B. Because security groups are stateful, the reply to an allowed request is permitted automatically on the way out.
2. The VPC router consults ewb-rt-b-private; 10.10.0.0/16 → pcx-0096559164ba7330a matches, so the reply is sent back through the peering connection.
3. In VPC A, the reply is delivered to App-A. Because the reply belongs to a request App-A initiated, App-A's security group allows it through connection tracking; the ICMP rule on ewb-sg-app-a (source 10.20.1.10/32) is needed only when App-B initiates the ping (the reverse test).
4. The ping command prints one reply line per packet; four replies give 0% packet loss.

13.3 Why Both Sides Need Routing and Security Rules

Connectivity requires every one of the following to be true at once: the peering connection is Active; the sender's subnet has a route to the destination CIDR through the peering connection; the destination's security group admits the traffic; and the destination's subnet has a route back to the sender's CIDR. If the forward route is missing, packets are never sent; if the return route is missing, requests arrive but replies are lost; if the security group does not allow the request, the destination silently drops it. This is why an Active peering status alone "does not guarantee a successful ping", and it is exactly what the controlled failure in Section 14 isolates: routes and peering stayed unchanged while a single security rule was removed.

14. Troubleshooting and Observations

14.1 Controlled ICMP Failure and Recovery

To show that the security group, and not only routing, controls reachability, the inbound ICMP rule on ewb-sg-app-b for source 10.10.2.10/32 was temporarily removed. App-A then pinged App-B at 10.20.1.10: all four packets were lost (4 transmitted, 0 received, 100% loss; Figure 17). The rule was then restored and the same ping run again in the same terminal session: 4 transmitted, 4 received, 0% loss (Figure 18). The SSH rule on ewb-sg-app-b was not part of this test.

- **What the result shows:** the peering connection stayed Active and the routes in both VPCs were unchanged throughout, so the only variable was the security group rule. Removing the allow rule was enough to stop the echo requests from being delivered, and restoring it was enough to bring connectivity back.
- **Why the ping gave no error message:** a security group drops unmatched inbound traffic silently. The sender receives no "unreachable" message, so ping simply reports 100% loss after its timeout (about 3.2 seconds for four packets here).
- **What the result does not show:** it does not show the pre-peering state (see the missing baseline in Section 16), and the removal and restoration of the rule were not themselves screenshotted; they are evidenced by their effect on the ping output and by the project record.

No other troubleshooting steps are claimed, because none are supported by the supplied evidence.

15. Results and Discussion

Requirement	Result	Evidence
Private connectivity between VPC A and VPC B	Achieved: bidirectional ping succeeded over private IPs with 0% loss.	Figs. 15, 16
Application instances remain private	Achieved: neither App-A nor App-B has a public IPv4 address; no 0.0.0.0/0 route in their route tables; VPC B has no IGW.	Figs. 3, 4, 7, 8
Forward and return routes	In place: 10.20.0.0/16 → pcx (VPC A tables) and 10.10.0.0/16 → pcx (VPC B).	Figs. 6–8
Least-privilege inbound rules	In place with one deviation: ICMP rules use "All ICMP – IPv4" rather than Echo request only.	Figs. 9–11
Administration without copying keys	Achieved: SSH to both private hosts through the bastion using ProxyJump.	Figs. 12–14
Controlled failure and recovery	Demonstrated: 100% loss with the rule removed; 0% loss after restoring it.	Figs. 17, 18
Cleanup	Lab instances and network resources removed after testing; final VPC list shows both lab VPCs absent. Key-pair removal and billing closure are not claimed.	Section 18; Appendix C cleanup evidence

16. Limitations

- **Missing baseline evidence.** The pre-peering ping failure (workbook Test 01) was not captured. This report therefore cannot demonstrate the "before" state of the network, and it does not claim to.

- **Subnet evidence.** No screenshot of the subnets or of the route tables' subnet associations was supplied, so subnet IDs and associations are not evidenced (Section 9.4). The workbook's Checkpoints 01 and 04 call for this evidence.
- **Rule change not screenshotted.** The removal and re-addition of the ICMP rule were not captured; the security group screenshots show the configured rules, and the failure and recovery are evidenced by the ping outputs.
- **ICMP rule type.** ICMP is permitted with "All ICMP – IPv4" rather than Echo request only, which is broader than the workbook specifies.
- **Outbound rules, tags and key pair.** Outbound security group rules, the Project/Student resource tags, the key pair name in AWS, the root volume's delete-on-termination setting and the SSH configuration file were not shown.
- **Single-AZ, single-account test.** All instances are in one AZ and one account and Region; results do not describe cross-AZ, cross-account or cross-Region peering.
- **Scope of ping.** ICMP and SSH were tested; no application-layer protocol between App-A and App-B was tested.
- Cleanup. Evidence was captured after terminating the three lab instances and deleting the lab peering connection, security groups, subnets, custom route tables, Internet Gateway and both lab VPCs. The final VPC list shows both lab VPCs absent; one other VPC remains. EBS volume evidence shows no volumes in the Sydney Region at the time of capture. Billing reporting can lag, and key-pair removal was not evidenced (Section 18).

17. Security Considerations

- **No credentials in the report or repository.** The report contains no private key content, passwords, access keys or tokens. Only the path/placeholder of the key file is mentioned. Account IDs and the operator's public IP were redacted from screenshots. Any repository holding this project should exclude *.pem files (for example via .gitignore).
- **Application instances remain private.** App-A and App-B have no public IPv4 address and no internet route; VPC B has no Internet Gateway.
- **Least-privilege security groups.** SSH to the bastion is limited to one operator /32, SSH to the private hosts to the bastion /32, and ICMP to the single peer /32. No rule uses 0.0.0.0/0. Narrowing ICMP to Echo request would tighten this further.
- **Safe handling of SSH keys.** The key pair is created once, stored only on the laptop with owner-only permissions, never uploaded or pasted onto an instance, and used via ProxyJump so the bastion never holds it. After the lab, the key pair should be deleted from AWS and the local file removed if no longer needed.
- **Host key verification.** The first connection to each host should be verified before accepting its host key.
- **Keep the bastion exposure short-lived.** A public bastion is the project's only internet-facing resource; it should be terminated promptly after testing, and the operator IP rule updated if the IP changes.

18. Cleanup and Cost Management

#	Cleanup action (workbook order)	Resource(s)	Status	Evidence
1	Save all evidence first	Screenshots and terminal output	[x] CAPTURED	06_Cleanup/01_vpc_inventory.png

#	Cleanup action (workbook order)	Resource(s)	Status	Evidence
2	Terminate the three instances	ewb-bastion i-047f321971d45e684; ewb-app-a i-0170f18f690c0ba41; ewb-app-b i-0b8529554ab35eb8d	[x]COMPLETED	06_Cleanup/10_instances_terminated.png
3	Confirm lab EBS volumes are deleted; delete any leftover volumes or snapshots	Root volumes of the 3 instances	[x] VERIFIED	06_Cleanup/11_storage_and_ips_cleanup.png shows no EBS volumes in the Region
4	Release any Elastic IP created	None shown for the bastion (Elastic IP field "–", Fig. 2); verify none exist	[x] VERIFIED	06_Cleanup/09_elastic_ips_before_cleanup.png shows no Elastic IPs allocated
5	Remove the unused key pair and the local .pem if no longer needed	EC2 key pair (workbook name ewb-lab) and local key file	[x] COMPLETED	Key-pair deletion/local .pem removal
6	Delete the peer routes	10.20.0.0/16 → pcx in ewb-rt-a-public and ewb-rt-a-private; 10.10.0.0/16 → pcx in ewb-rt-b-private	[x] COMPLETED	06_Cleanup/12a_routes_public_after_removal.png; 12b_routes_private_a_after_removal.png; 12c_routes_private_b_after_removal.png
7	Delete the peering connection	ewb-a-b-peer, pcx-0096559164ba7330a	[x] COMPLETED	06_Cleanup/13_peering_deleted.png
8	Delete the lab security groups (after instance network interfaces are gone)	sg-0725cba0ffaaefc14, sg-02e3f9a604ebb0522, sg-0ce9e1b7264e57a37	[x] COMPLETED	06_Cleanup/14_security_groups_deleted.png
9	Remove subnet associations, then delete the lab subnets	ewb-a-public, ewb-a-private, ewb-b-private (IDs to be filled in)	[x] COMPLETED	06_Cleanup/15_subnets_deleted.png
10	Delete the custom route tables	rtb-05928fa32e5e99a75, rtb-0e32ae5a45c32e421, rtb-019b95b5bfdda447a	[x] COMPLETED	06_Cleanup/16_custom_route_tables_deleted.png
11	Detach and delete the Internet Gateway	ewb-igw-a, igw-0596b63f8cc23aeef	[x] COMPLETED	06_Cleanup/17_internet_gateway_deleted.png
12	Delete both VPCs	vpc-003581adc7d95c74d; vpc-0832d4c7ef00b2189	[x] COMPLETED	06_Cleanup/18_vpcs_deleted.png

#	Cleanup action (workbook order)	Resource(s)	Status	Evidence
13	Verify the account is clear	Filter by project tag in ap-southeast-2: EC2 instances, volumes, public IPv4 allocations, VPC resources; check Billing (usage reporting can lag); capture evidence	[x] CAPTURED	06_Cleanup/19_final_cleanup_verification.png; billing check not evidenced and may lag

18.1 Cost Management Notes

- EC2 instance hours, EBS volumes and public IPv4 addresses can all incur charges; confirm current pricing and account credits before launching.
- This lab needs no NAT gateway, which avoids one of the most common hidden costs in VPC labs.
- The bastion is the only resource with a public IPv4 address; terminating it first stops the public IPv4 charge.
- Keep instances only for as long as testing needs them; stopped instances still pay for their EBS volumes.
- Delete in the dependency order above, because AWS refuses to delete a security group, subnet or VPC while dependent resources still exist.

19. Conclusion and What I Learned

The project achieved private, two-way connectivity between two non-overlapping VPCs in the Sydney Region while keeping both application instances free of public IPv4 addresses. Administration was possible only through a single, narrowly restricted bastion, using SSH ProxyJump so that no private key was stored on any server. The controlled failure demonstrated that connectivity depends on layers working together: peering, routing in both directions and a security rule at the destination.

What I learned.

- An Active peering connection is only a link; routes on both sides and a permissive security rule decide whether traffic actually flows.
- Routes are directional, so the return route is as important as the forward route.
- A bastion using ProxyJump means the target sees the bastion's private IP, which is why the SSH rule source is the bastion's address and not the laptop's.
- Peering is non-transitive and gives no edge-to-edge routing, so VPC B cannot use VPC A's Internet Gateway.
- Evidence discipline matters: capturing the baseline failure, the subnet associations and the rule change at the time would have made the report complete. These gaps are documented honestly here instead of being filled with assumptions.

20. References

- EWB Courses, *AWS VPC Peering: Student Project Workbook* (Project 01, AWS Architect Associate, EWB Cloud Labs), October 2026.
- AWS documentation (docs.aws.amazon.com), as listed in the workbook: VPC peering concepts and limitations; Create a VPC peering connection; Update route tables for peering; Security groups for peered VPCs; EC2 key pair security; Amazon VPC pricing. The workbook states these were checked on 16 October 2026.

- Amazon Web Services, Amazon VPC User Guide: subnets, route tables, Internet gateways and security groups (docs.aws.amazon.com/vpc).
- OpenSSH manual pages: ssh_config(5) (ProxyJump) and ssh(1) (-J).

Appendix A. Evidence Index

Fig.	File (evidence folder)	Shows	Redaction
2	05_EC2_Private_IPs/01_Bastion_IP.png	Bastion instance details	None
3	05_EC2_Private_IPs/02_AppA_Private_IP.png	App-A instance details	None
4	05_EC2_Private_IPs/03_AppB_Private_IP.png	App-B instance details	None
5	03_VPC_Peering_and_Routes/01_Peering_Active.png	Peering connection Active	Account IDs and ARN
6	03_VPC_Peering_and_Routes/04_Route_A_Public.png	ewb-rt-a-public routes	None
7	03_VPC_Peering_and_Routes/02_Route_A_Private.png	ewb-rt-a-private routes	None
8	03_VPC_Peering_and_Routes/03_Route_B_Private.png	ewb-rt-b-private routes	None
9	04_Security_Groups/01_SG_Bastion.png	ewb-sg-bastion inbound rule	Operator public IP
10	04_Security_Groups/02_SG_AppA.png	ewb-sg-app-a inbound rules	None
11	04_Security_Groups/03_SG_AppB.png	ewb-sg-app-b inbound rules	None
12	02_SSH_Access/03_SSH_Bastion.png	SSH to bastion	Previous-login source IP
13	02_SSH_Access/01_SSH_AppA.png	SSH to App-A via bastion	None
14	02_SSH_Access/02_SSH_AppB.png	SSH to App-B via bastion	None
15	01_Ping_Tests/01_AppA_to_AppB_Success.png	Forward ping	None
16	01_Ping_Tests/02_AppB_to_AppA_Success.png	Reverse ping	None
17	01_Ping_Tests/03_Controlled_Failure.png	Controlled ICMP failure	None
18	01_Ping_Tests/04_Connectivity_Restored.png	Failure then recovery	None

Figure 1 is the architecture diagram drawn for this report; it is not a screenshot.

Appendix B. Workbook Submission Checklist Mapping

Workbook requirement (page 13)	Where addressed	Status
Cover: name, batch, title, Region, completion date	Cover page	Complete
Architecture diagram and inventory with actual IDs and private IPs	Sections 6 and 7	Complete
Subnet CIDRs and all three route tables, including subnet associations	Sections 7 and 9; Appendix C	Complete

Workbook requirement (page 13)	Where addressed	Status
Active peering connection and inbound rules for each security group	Sections 8 and 10	Complete
Baseline failure, forward/reverse ping success, App-B SSH proof	Sections 11 and 12	Complete
Controlled ICMP failure and recovery with short explanation	Sections 12 and 14	Complete
Cleanup evidence plus a short note on what was learned	Sections 18 and 19; Appendix C	Complete
Forward and return path explanation; no keys, passwords or tokens	Section 13; Section 17	Complete

Appendix C. Cleanup and Supplemental AWS Console Evidence

The following figures reproduce the screenshots supplied for this report. They document the pre-cleanup inventory and associations, the relevant route and security-group state, and the cleanup actions and final verification. Screenshots are evidence of the console state at the time captured; they do not independently establish billing closure or removal of the local private-key file.

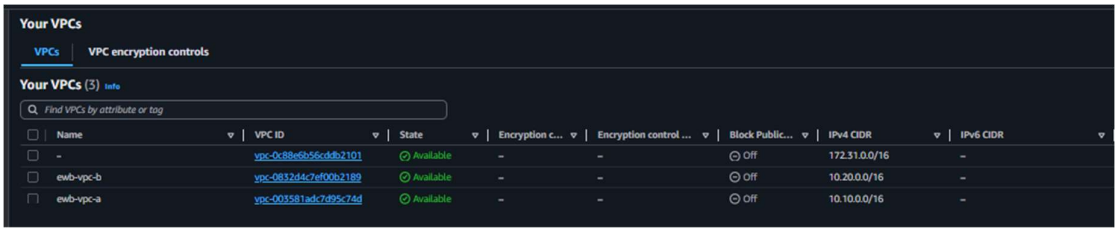


Figure C1. Supplemental VPC inventory captured before cleanup.

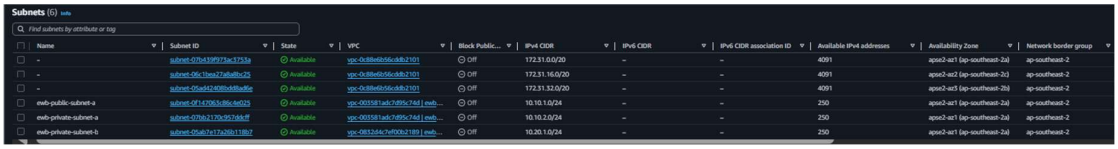


Figure C2. Subnet inventory showing the lab subnet CIDR ranges.

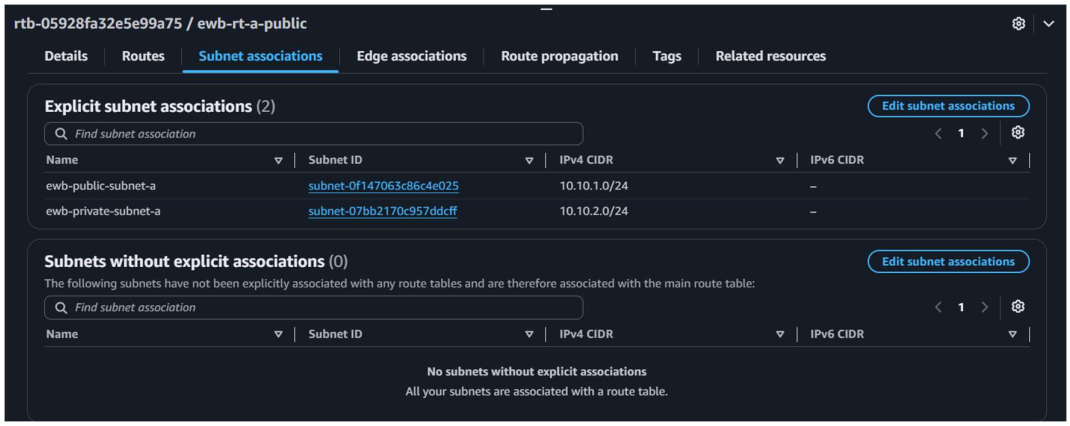


Figure C3. Public route-table subnet association evidence.

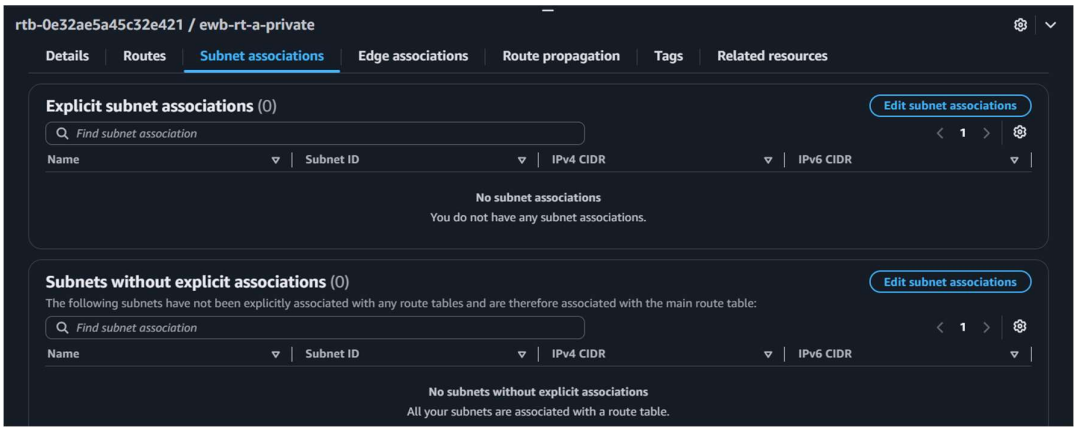


Figure C4. Private route table A subnet association evidence.

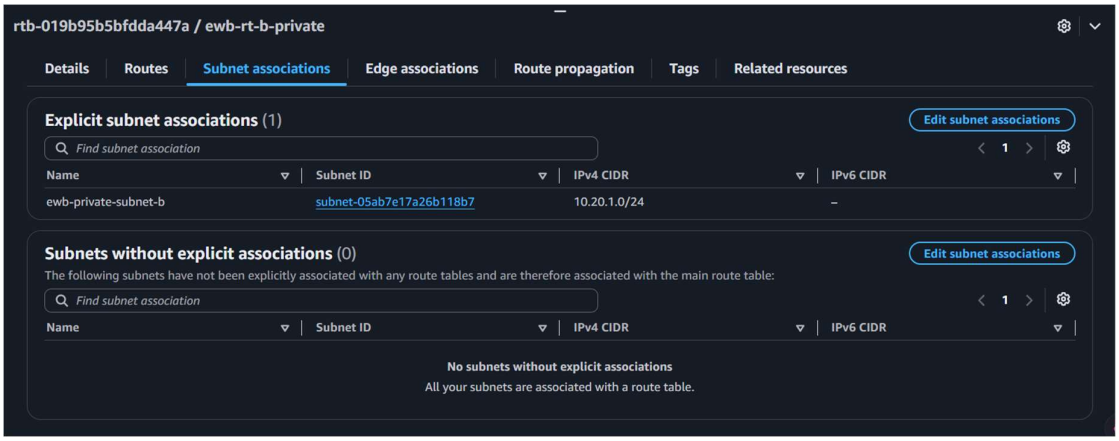


Figure C5. Private route table B subnet association evidence.

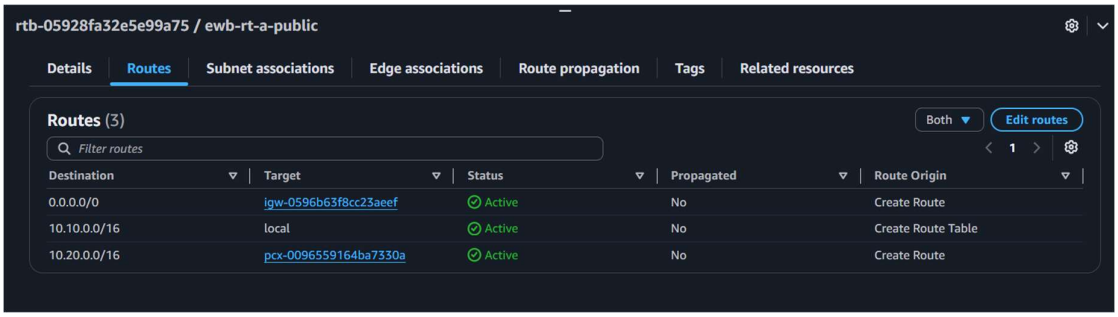


Figure C6. Public route-table routes before cleanup.

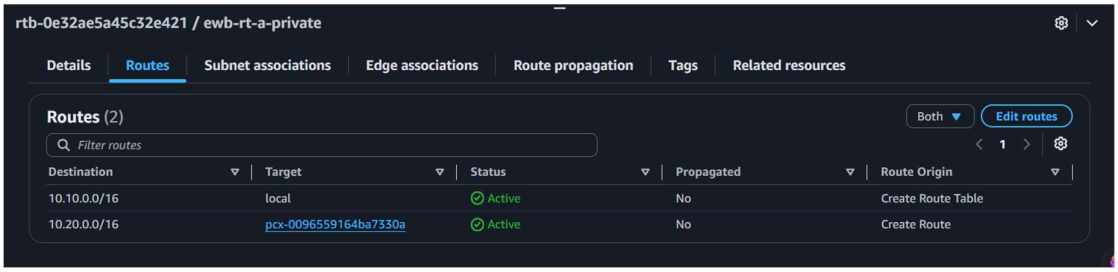


Figure C7. Private route table A routes before cleanup.

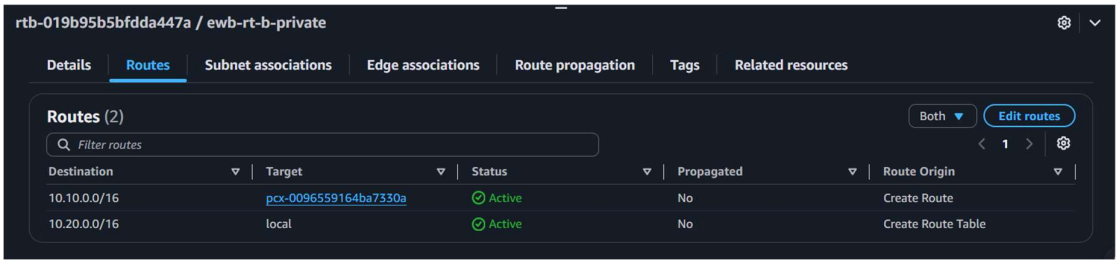


Figure C8. Private route table B routes before cleanup.

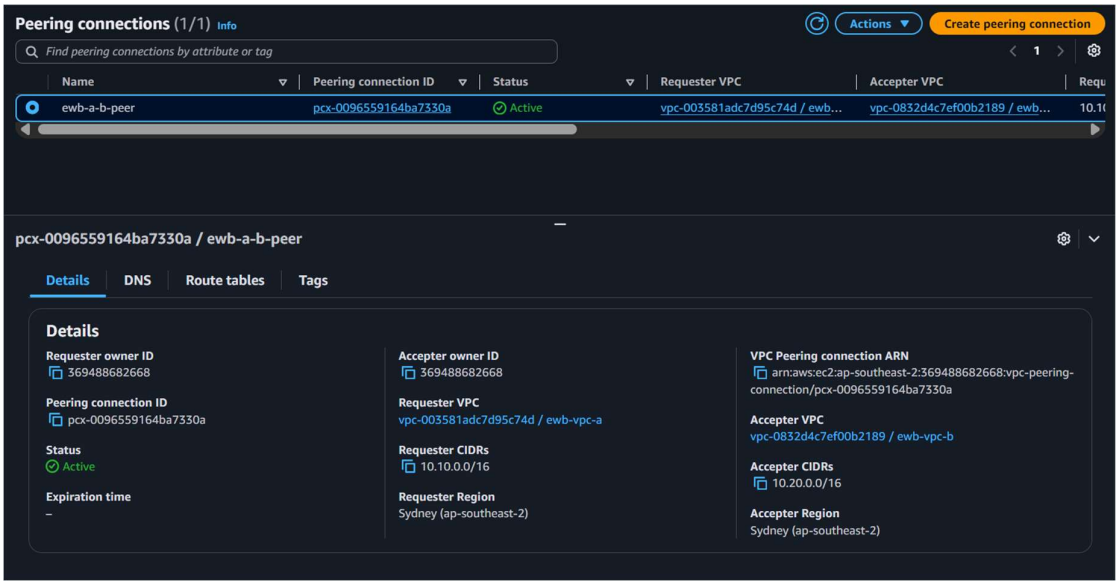


Figure C9. VPC peering connection shown in the active testing phase.

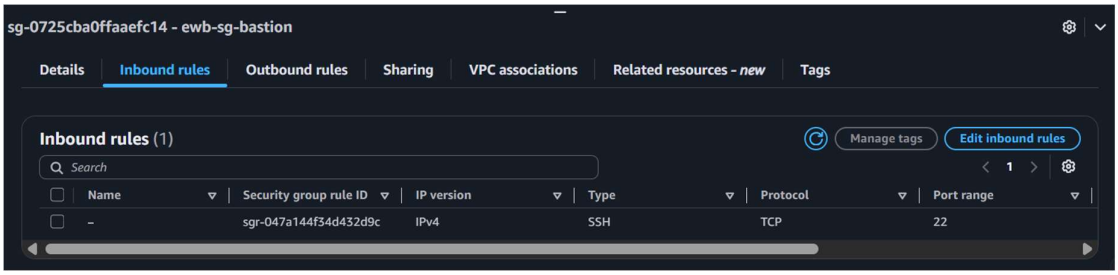


Figure C10. Bastion security-group configuration before cleanup.



Figure C11. Application A security-group configuration before cleanup.

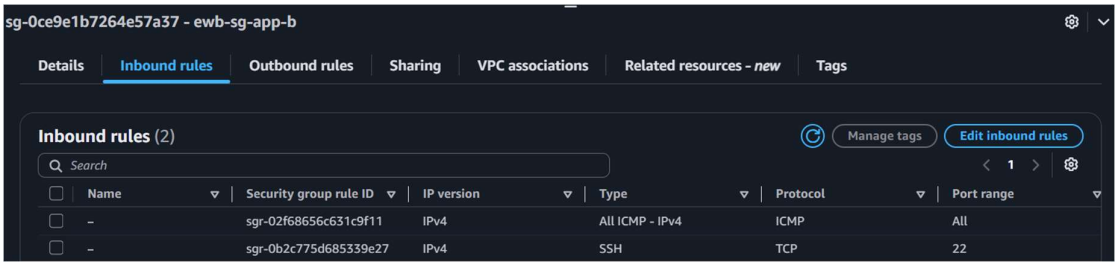


Figure C12. Application B security-group configuration before cleanup.

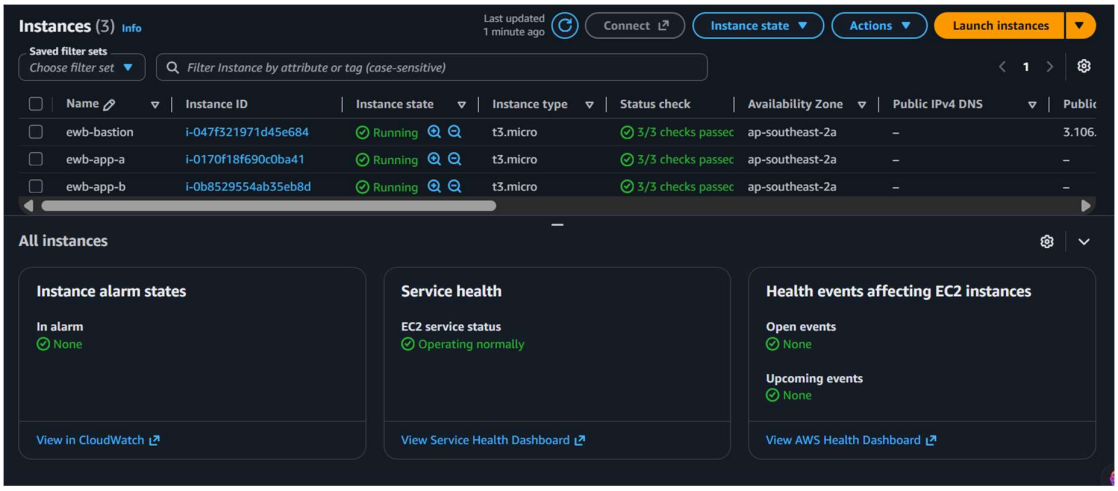


Figure C13. EC2 lab inventory before cleanup.

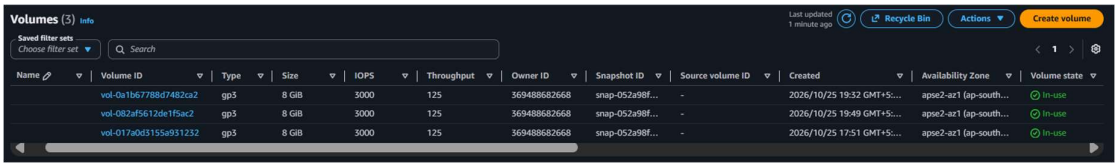


Figure C14. EBS volume inventory before cleanup.

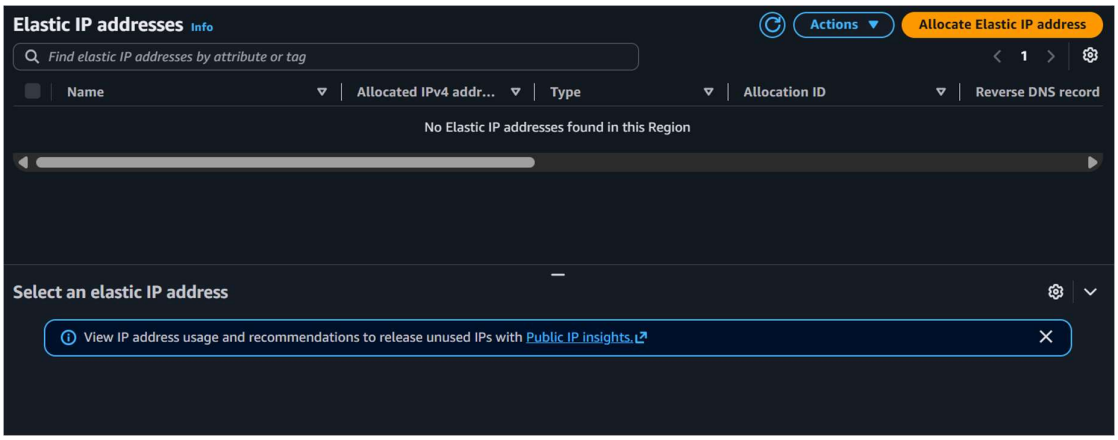


Figure C15. Elastic IP inventory; no Elastic IPs are shown as allocated.

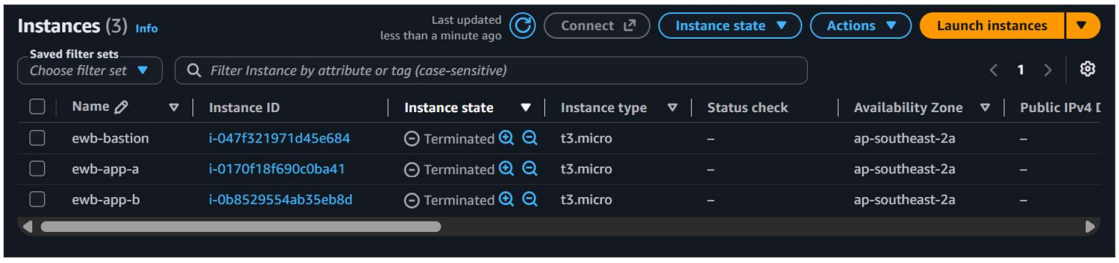


Figure C16. Lab EC2 instances after termination.

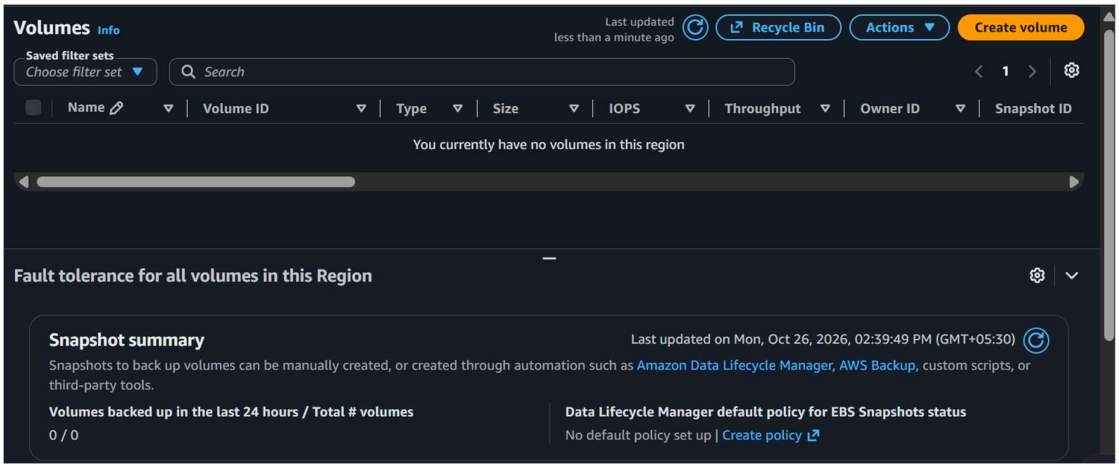


Figure C17. Post-cleanup EBS Volumes page showing no volumes in the Sydney Region.

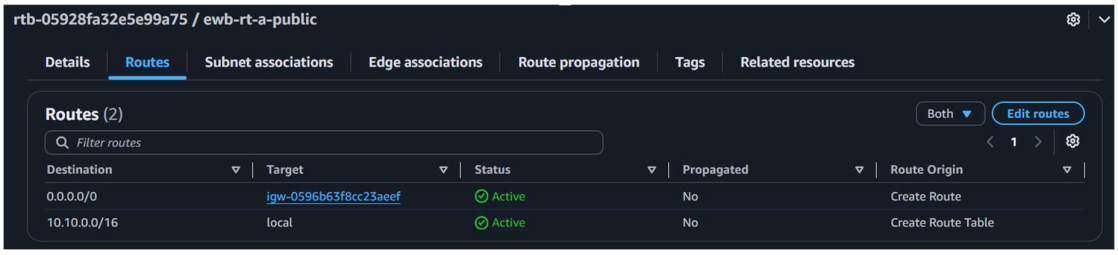


Figure C18. Public route table after removal of any route targeting the lab peering connection.

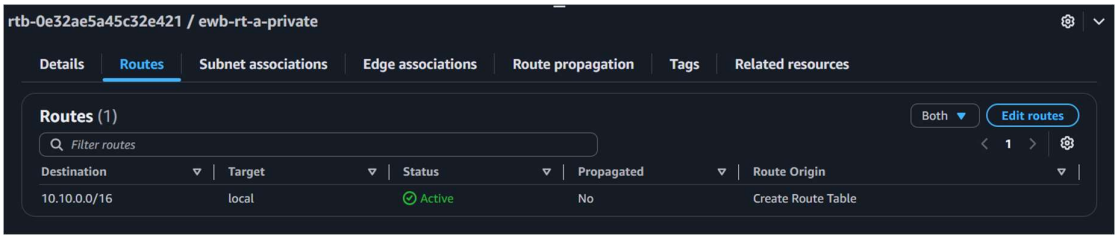


Figure C19. Private route table A after removal of the route targeting the lab peering connection.

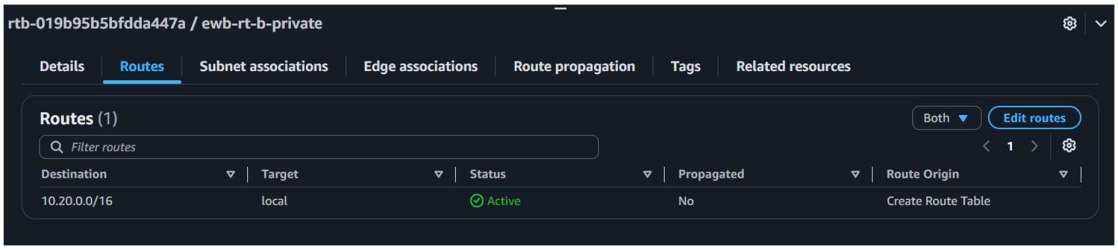


Figure C20. Private route table B after removal of the route targeting the lab peering connection.

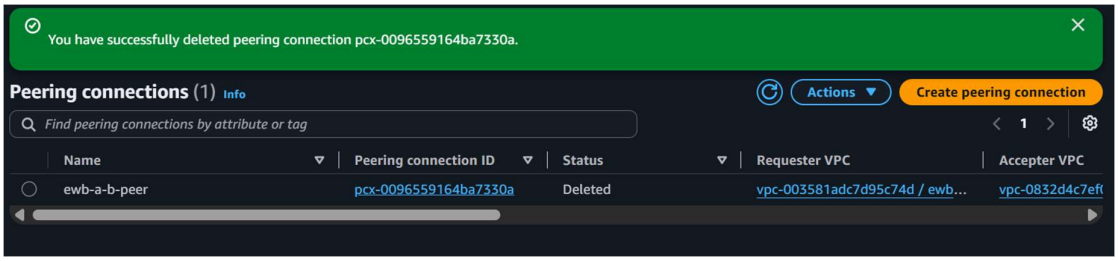


Figure C21. Peering connections page after deletion of the lab peering connection.

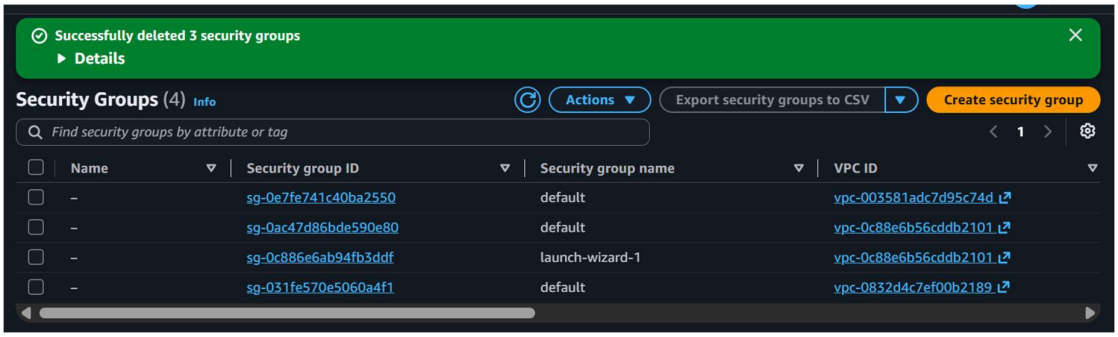


Figure C22. Security-group console evidence after lab security-group deletion.

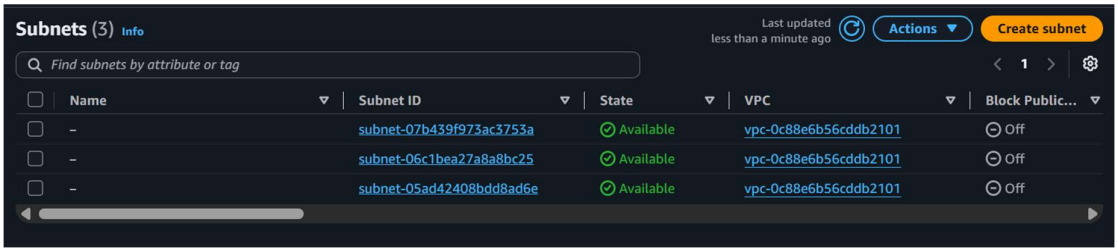


Figure C23. Subnets console evidence after lab subnet deletion.

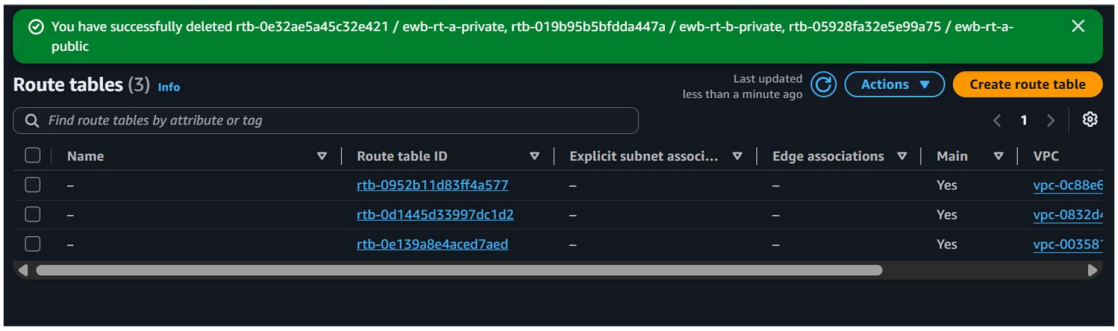


Figure C24. Route-table console evidence after lab custom route-table deletion.

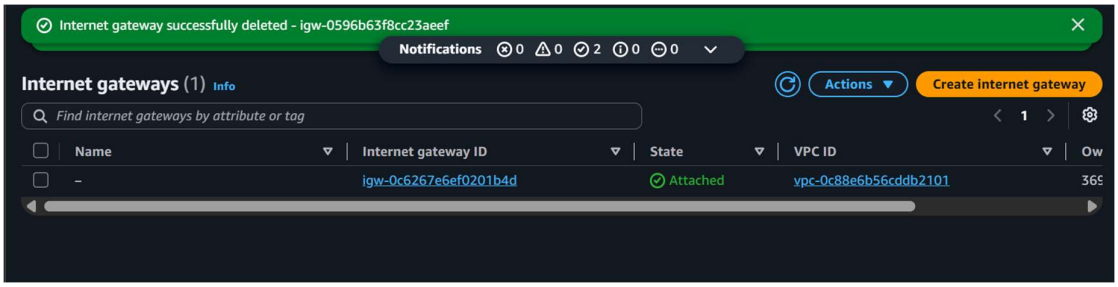


Figure C25. Internet Gateways page after deletion of the lab Internet Gateway.

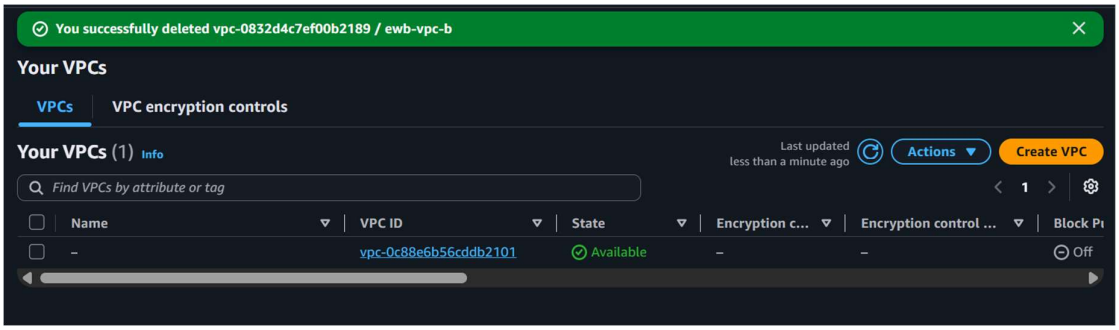


Figure C26. VPC inventory after deletion of the two lab VPCs.

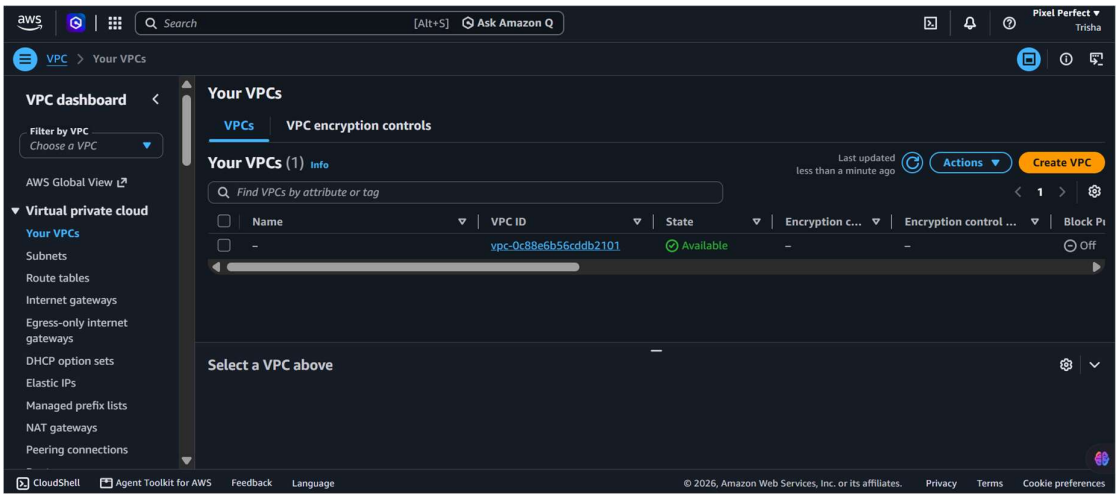


Figure C27. Final VPC list: ewb-vpc-a and ewb-vpc-b are absent; one other VPC remains.